

Complete DSA & Competitive Programming Guide in C++

For Beginners Who Know Basic C++ Syntax

How to use this guide: Read topics in order. Each topic builds on previous ones. For every topic: read the explanation, trace through the code manually, then solve the practice problems.

TABLE OF CONTENTS

1. Time Complexity & Big O
2. Arrays
3. Strings
4. Vectors
5. Sorting
6. Binary Search
7. Prefix Sums
8. Frequency Arrays
9. Two Pointers
10. Sliding Window
11. Hash Maps
12. Sets
13. Stacks
14. Queues
15. Deques
16. Linked Lists
17. Recursion
18. Backtracking
19. Trees
20. Binary Search Trees
21. Heaps / Priority Queues
22. Graph Fundamentals
23. DFS
24. BFS
25. Topological Sort
26. Disjoint Set Union (Union-Find)
27. Greedy Algorithms

28. Bit Manipulation
 29. Dynamic Programming
 30. Shortest Path Algorithms
 31. Minimum Spanning Trees
 32. Segment Trees
 33. Sparse Tables
 34. Common Competitive Programming Patterns
 35. Common CSES Patterns
 36. Common Codeforces Patterns
- 100 Most Important STL Functions
 - Time Complexity Cheat Sheet
 - Algorithm Selection Cheat Sheet
-

BOILERPLATE TEMPLATE

Every competitive programming file should start with this:

```
cpp
```

```

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef pair<int,int> pii;
typedef vector<int> vi;
typedef vector<ll> vll;

#define all(x) (x).begin(), (x).end()
#define pb push_back
#define fi first
#define se second

const int INF = 1e9;
const ll LLINF = 1e18;
const int MOD = 1e9 + 7;

int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    // your code here

    return 0;
}

```

Why `ios_base::sync_with_stdio(false)` and `cin.tie(NULL)`? These two lines speed up I/O by a factor of 5-10x. Always include them.

1. TIME COMPLEXITY & BIG O

What It Is

Big O notation describes how the runtime (or memory) of an algorithm grows as input size n increases. It describes the **worst case** unless stated otherwise.

Why It Matters

In competitive programming, you're given time limits (usually 1-2 seconds). A too-slow algorithm will get **Time Limit Exceeded (TLE)**. You must estimate whether your algorithm is fast enough **before coding**.

The Golden Rule: Operations per second

A modern computer does roughly 10^8 simple operations per second.

n (input size)	Max acceptable complexity
$n \leq 10$	$O(n!)$ or $O(2^n)$
$n \leq 20$	$O(2^n)$
$n \leq 100$	$O(n^3)$
$n \leq 1,000$	$O(n^2)$
$n \leq 100,000$	$O(n \log n)$
$n \leq 1,000,000$	$O(n)$
$n \leq 10^{18}$	$O(\log n)$ or $O(\sqrt{n})$

Common Complexities (Fastest to Slowest)

$O(1)$	Constant	Array indexing, hash map lookup
$O(\log n)$	Logarithmic	Binary search, balanced BST ops
$O(\sqrt{n})$	Square root	Factorization, sieve optimizations
$O(n)$	Linear	Single loop through array
$O(n \log n)$	Linearithmic	Merge sort, heap sort, most sorting
$O(n^2)$	Quadratic	Nested loops, bubble sort
$O(n^3)$	Cubic	Triple nested loops, naive matrix multiply
$O(2^n)$	Exponential	All subsets, brute force with recursion
$O(n!)$	Factorial	All permutations

How to Calculate Big O

Rule 1: Drop constants. $O(3n) \rightarrow O(n)$. **Rule 2:** Drop lower-order terms. $O(n^2 + n) \rightarrow O(n^2)$. **Rule 3:** Loops multiply. A loop inside a loop $\rightarrow O(n^2)$. **Rule 4:** Sequential blocks add (and then dominated by the largest).

cpp

```

// O(n): single loop
for(int i = 0; i < n; i++) { ... }

// O(n^2): nested loops
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++) { ... }

// O(n * m): nested loops with different bounds
for(int i = 0; i < n; i++)
    for(int j = 0; j < m; j++) { ... }

// O(n + m): sequential loops
for(int i = 0; i < n; i++) { ... }
for(int j = 0; j < m; j++) { ... }

// O(log n): halving the problem each step
while(n > 1) { n /= 2; }

// O(n log n): loop with binary search inside
for(int i = 0; i < n; i++) {
    binary_search(...); // O(log n)
}

```

Space Complexity

Same notation, but for memory usage.

- $O(1)$: uses fixed extra memory regardless of input
- $O(n)$: uses memory proportional to input size

Common Mistakes

- Forgetting that recursion uses $O(\text{depth})$ stack space
- Thinking $O(n \log n)$ is always fine — check $n \leq 10^6$, not 10^9
- Ignoring constant factors when n is borderline

2. ARRAYS

What It Is

A fixed-size, contiguous block of memory storing elements of the same type. The most fundamental data structure.

Why It Matters

Arrays underlie almost everything. Direct index access is $O(1)$. Cache-friendly (memory locality).

When to Use

- When size is known at compile time
- When you need $O(1)$ random access
- For frequency counting, prefix sums, DP tables

Time Complexity

Operation	Complexity
Access by index	$O(1)$
Search (unsorted)	$O(n)$
Search (sorted)	$O(\log n)$
Insert at end	$O(1)$
Insert at middle	$O(n)$
Delete at middle	$O(n)$

Key STL Functions

```
cpp
```

```
int arr[5] = {3, 1, 4, 1, 5};

// Sort
sort(arr, arr+5);           // {1, 1, 3, 4, 5}

// Binary search (array must be sorted)
binary_search(arr, arr+5, 3); // returns true

// Fill
fill(arr, arr+5, 0);        // all zeros

// Min/Max element
*min_element(arr, arr+5);    // returns min value
*max_element(arr, arr+5);    // returns max value

// Sum
accumulate(arr, arr+5, 0);   // sum of all elements

// Reverse
reverse(arr, arr+5);
```

Example Code: Common Array Operations

cpp

```

#include <bits/stdc++.h>
using namespace std;

int main(){
    int n;
    cin >> n;
    int arr[n];

    // Read array
    for(int i = 0; i < n; i++) cin >> arr[i];

    // Find max and its index
    int maxVal = arr[0], maxIdx = 0;
    for(int i = 1; i < n; i++){
        if(arr[i] > maxVal){
            maxVal = arr[i];
            maxIdx = i;
        }
    }

    // Count occurrences of x
    int x = 3, count = 0;
    for(int i = 0; i < n; i++)
        if(arr[i] == x) count++;

    // Rotate right by 1: {1,2,3,4,5} -> {5,1,2,3,4}
    int temp = arr[n-1];
    for(int i = n-1; i > 0; i--) arr[i] = arr[i-1];
    arr[0] = temp;

    // Check if sorted
    bool isSorted = true;
    for(int i = 1; i < n; i++)
        if(arr[i] < arr[i-1]) { isSorted = false; break; }

    return 0;
}

```

2D Arrays

cpp


```

int mat[3][4]; // 3 rows, 4 cols

// Read 2D array
for(int i = 0; i < 3; i++)
    for(int j = 0; j < 4; j++)
        cin >> mat[i][j];

// Traverse row by row (cache-friendly)
for(int i = 0; i < 3; i++)
    for(int j = 0; j < 4; j++)
        cout << mat[i][j] << " ";

// Diagonal elements (only for square matrix)
for(int i = 0; i < n; i++) cout << mat[i][i]; // main diagonal
for(int i = 0; i < n; i++) cout << mat[i][n-1-i]; // anti-diagonal

```

Common Mistakes

- Array index out of bounds (no automatic check in C++ — causes undefined behavior)
- Using VLAs (variable-length arrays) — prefer vectors for dynamic sizes
- Off-by-one errors: `arr[n]` doesn't exist, last valid index is `arr[n-1]`

Practice Problems

- Codeforces 4A: Watermelon
- CSES: Weird Algorithm
- Leetcode 1: Two Sum (brute force version)
- Leetcode 283: Move Zeroes

3. STRINGS

What It Is

A sequence of characters. In C++, use `std::string` (not char arrays) in competitive programming.

Why It Matters

Many problems involve string parsing, pattern matching, character frequency counting.

Time Complexity

Operation	Complexity
Access by index	$O(1)$
Length	$O(1)$
Concatenation (+)	$O(n)$
Substring	$O(n)$
Find/search	$O(n*m)$ naive
Comparison	$O(n)$

Key STL Functions

```
cpp
```

```

string s = "hello world";

s.size()           // or s.length() – both return 11
s[0]               // 'h'
s.substr(6, 5)      // "world" – substr(start, length)
s.find("world")     // returns 6 (index), or string::npos if not found
s.rfind('l')        // last occurrence index = 9
s.replace(0, 5, "bye") // "bye world"
s.insert(5, " there") // "hello there world"
s.erase(5, 6)       // erase 6 chars starting at index 5
s.push_back('!')    // append character
s.pop_back()        // remove last character
s.empty()           // returns true if length is 0
s.clear()           // empties the string

// Conversions
to_string(42)       // int to string
stoi("42")          // string to int
stoll("1234567890") // string to long long
stod("3.14")        // string to double

// Case conversion
for(char &c : s) c = toupper(c); // or tolower(c)

// Check character type
isalpha('a') // true – is letter
isdigit('5') // true – is digit
isspace(' ') // true – is whitespace
islower('a') // true
isupper('A') // true

```

Common Patterns

Pattern 1: Read line with spaces

```

cpp

string line;
getline(cin, line); // reads entire line including spaces

```

Pattern 2: Character frequency

```

cpp

```

```
string s;
cin >> s;
int freq[26] = {0};
for(char c : s) freq[c - 'a']++; // lowercase letters only
```

Pattern 3: Parse words from string

```
cpp

string line;
getline(cin, line);
stringstream ss(line);
string word;
vector<string> words;
while(ss >> word) words.push_back(word);
```

Pattern 4: Palindrome check

```
cpp

bool isPalindrome(string s){
    int l = 0, r = s.size() - 1;
    while(l < r){
        if(s[l] != s[r]) return false;
        l++; r--;
    }
    return true;
}
```

Pattern 5: String reversal

```
cpp

string s = "hello";
reverse(s.begin(), s.end()); // "olleh"
// Or: string rev(s.rbegin(), s.rend());
```

Pattern 6: Split by delimiter

```
cpp
```

```

string s = "a,b,c,d";
vector<string> parts;
stringstream ss(s);
string part;
while(getline(ss, part, ',')) parts.push_back(part);
// parts = ["a", "b", "c", "d"]

```

Example Code: Anagram Check

```

cpp

bool isAnagram(string a, string b){
    if(a.size() != b.size()) return false;
    int freq[26] = {0};
    for(char c : a) freq[c-'a']++;
    for(char c : b) freq[c-'a']--;
    for(int i = 0; i < 26; i++)
        if(freq[i] != 0) return false;
    return true;
}

```

Common Mistakes

- `s.find()` returns `string::npos` (a large unsigned number) when not found — always check with `!= string::npos`
- Comparing `char` with `int` without explicit cast can cause issues
- `s.substr(pos, length)` — the second argument is LENGTH, not end index
- String concatenation in a loop is $O(n^2)$ — use stringstream or join at end

Practice Problems

- CSES: Permutations (understand indices)
- Leetcode 242: Valid Anagram
- Leetcode 344: Reverse String
- Leetcode 14: Longest Common Prefix
- Codeforces 71A: Way Too Long Words

4. VECTORS

What It Is

A dynamic array that automatically resizes. The most-used container in competitive programming.

Why It Matters

Vectors give you the convenience of dynamic sizing with nearly the same performance as raw arrays.

Time Complexity

Operation	Complexity
Access by index	$O(1)$
push_back	$O(1)$ amortized
pop_back	$O(1)$
Insert at middle	$O(n)$
Erase at middle	$O(n)$
Size	$O(1)$
Clear	$O(n)$

Key STL Functions

```
cpp
```

```

vector<int> v;
vector<int> v(5, 0);           // size 5, all zeros
vector<int> v = {1,2,3,4,5};
vector<int> v(arr, arr+n);    // from array

v.push_back(10);             // add to end
v.pop_back();                 // remove from end
v.size();                     // number of elements
v.empty();                    // true if no elements
v.clear();                     // remove all elements
v.front();                     // first element
v.back();                     // last element
v[i];                          // access by index (no bounds check)
v.at(i);                       // access by index (with bounds check – throws exception)
v.begin();                     // iterator to first element
v.end();                       // iterator past last element
v.resize(10);                  // resize to 10 elements (new ones are 0)
v.resize(10, -1);              // resize to 10 elements (new ones are -1)
v.reserve(100);                // reserve memory for 100 (avoids reallocation)
v.assign(5, 0);                // set 5 elements all to 0
v.insert(v.begin()+2, 99);     // insert 99 at index 2
v.erase(v.begin()+2);          // erase element at index 2
v.erase(v.begin()+1, v.begin()+3); // erase range [1,3)

// Sorting
sort(v.begin(), v.end());      // ascending
sort(v.begin(), v.end(), greater<int>()); // descending

// Algorithms
auto it = find(v.begin(), v.end(), 5); // find value 5
int idx = lower_bound(v.begin(), v.end(), 5) - v.begin(); // binary search
*min_element(v.begin(), v.end());
*max_element(v.begin(), v.end());
accumulate(v.begin(), v.end(), 0LL); // sum (use 0LL for long long)
reverse(v.begin(), v.end());
unique(v.begin(), v.end());          // remove consecutive duplicates
count(v.begin(), v.end(), 3);        // count occurrences of 3

```

2D Vectors

```
cpp
```

```
// n rows, m cols, all zeros
vector<vector<int>> grid(n, vector<int>(m, 0));

// Access
grid[i][j] = 5;

// Add a row
grid.push_back(vector<int>(m, 0));

// Adjacency list for graph
vector<vector<int>> adj(n);
adj[u].push_back(v);
adj[v].push_back(u);
```

Common Patterns

Pattern: Read n elements into vector

```
cpp

int n; cin >> n;
vector<int> v(n);
for(int i = 0; i < n; i++) cin >> v[i];
// Or: for(auto &x : v) cin >> x;
```

Pattern: Deduplicate a sorted vector

```
cpp

sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
```

Pattern: Create index array sorted by value

```
cpp

vector<int> idx(n);
iota(idx.begin(), idx.end(), 0); // idx = {0,1,2,...,n-1}
sort(idx.begin(), idx.end(), [&](int a, int b){
    return v[a] < v[b];
});
```

Common Mistakes

- Using `v.size() - 1` when v is empty (size() returns unsigned, so it wraps to a huge number)

- Iterating with `int i = 0; i < v.size()` — `v.size()` returns `size_t` (unsigned); use `((int)v.size())` or just `i < (int)v.size()`
- Invalidating iterators by `push_back` inside a loop over the same vector

Practice Problems

- CSES: Two Sets
 - Leetcode 26: Remove Duplicates from Sorted Array
 - Leetcode 189: Rotate Array
-

5. SORTING

What It Is

Rearranging elements in order (ascending or descending). One of the most fundamental operations.

Why It Matters

Sorting is a prerequisite for binary search, greedy algorithms, and many DP optimizations. Many problems become trivially easy after sorting.

When to Use

- When you need to find min/max efficiently
- When you need to binary search
- When processing elements in a specific order helps
- When you need to find duplicates or pairs

Time Complexity

Algorithm	Best	Average	Worst	Space
<code>std::sort</code>	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Counting sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$

`std::sort` in C++ uses introsort (hybrid of quicksort + heapsort + insertion sort). Always use it.

Key STL Functions

cpp

```
vector<int> v = {5, 2, 8, 1, 9};

// Basic sort (ascending)
sort(v.begin(), v.end());

// Descending
sort(v.begin(), v.end(), greater<int>());

// Custom comparator
sort(v.begin(), v.end(), [](int a, int b){
    return a > b; // descending
});

// Sort pairs: by first, then second
vector<pair<int,int>> pairs;
sort(pairs.begin(), pairs.end()); // default: by first, then second

// Sort strings by length
vector<string> words;
sort(words.begin(), words.end(), [](const string& a, const string& b){
    return a.size() < b.size();
});

// Partial sort: get k smallest in sorted order
partial_sort(v.begin(), v.begin()+k, v.end());

// nth_element: O(n) – puts kth element in correct position
nth_element(v.begin(), v.begin()+k, v.end());

// stable_sort: preserves relative order of equal elements
stable_sort(v.begin(), v.end());

// is_sorted check
is_sorted(v.begin(), v.end());

// Check if sorted with custom comparator
is_sorted(v.begin(), v.end(), greater<int>());
```

Sorting Structs

```
cpp

struct Student {
    string name;
    int grade;
    int age;
};

vector<Student> students;

// Sort by grade descending, then name ascending
sort(students.begin(), students.end(), [](const Student& a, const Student& b){
    if(a.grade != b.grade) return a.grade > b.grade;
    return a.name < b.name;
});

// Alternatively, define operator< in struct
bool operator<(const Student& other) const {
    return grade > other.grade;
}
```

Counting Sort (for small integer ranges)

```
cpp

// Sort array where all values are in [0, MAXVAL)
void countingSort(vector<int>& arr, int maxVal){
    vector<int> count(maxVal, 0);
    for(int x : arr) count[x]++;
    int idx = 0;
    for(int val = 0; val < maxVal; val++)
        while(count[val]--) arr[idx++] = val;
}

// Time: O(n + maxVal), Space: O(maxVal)
```

Common Patterns

Pattern: Sort and find closest pair

```
cpp

sort(v.begin(), v.end());
int minDiff = INT_MAX;
for(int i = 1; i < v.size(); i++)
    minDiff = min(minDiff, v[i] - v[i-1]);
```

Pattern: Sort by multiple criteria

```
cpp
// Sort intervals by start, then by end if starts are equal
sort(intervals.begin(), intervals.end(), [](auto& a, auto& b){
    return a[0] != b[0] ? a[0] < b[0] : a[1] < b[1];
});
```

Common Mistakes

- Forgetting to sort before binary search
- Using wrong comparator (returning $\leq$ instead of $<$ causes undefined behavior)
- Sorting pairs when you want to sort by index (sort an index array instead)

Practice Problems

- CSES: Sorting and Searching - Distinct Numbers
 - Leetcode 56: Merge Intervals
 - Leetcode 179: Largest Number
 - Codeforces 500A: New Year Transportation
-

6. BINARY SEARCH

What It Is

An algorithm to find a value in a **sorted** array in $O(\log n)$ time by repeatedly halving the search space.

Why It Matters

Binary search is everywhere. The key insight: whenever you can determine "too high" or "too low", you can binary search.

When to Use

- Searching in sorted array
- Finding the first/last position of a value
- "Minimize the maximum" or "maximize the minimum" problems
- Finding the answer in a monotone search space (binary search on answer)

Time Complexity

- $O(\log n)$ for all operations

The Template

There are multiple ways to write binary search. Here is one clean template:

```
cpp
// Standard: find exact value
int binarySearch(vector<int>& arr, int target){
    int lo = 0, hi = arr.size() - 1;
    while(lo <= hi){
        int mid = lo + (hi - lo) / 2; // avoid overflow!
        if(arr[mid] == target) return mid;
        else if(arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1; // not found
}
```

Why $(lo + (hi - lo) / 2)$ not $(lo + hi) / 2$? Because $(lo + hi)$ can overflow when both are large ints.

STL Binary Search Functions (sorted containers only)

```
cpp
vector<int> v = {1, 2, 3, 4, 5, 5, 5, 7};

// Does value exist?
binary_search(v.begin(), v.end(), 5); // true

// First position >= target (lower bound)
auto it = lower_bound(v.begin(), v.end(), 5);
int idx = it - v.begin(); // idx = 4 (first 5)

// First position > target (upper bound)
auto it2 = upper_bound(v.begin(), v.end(), 5);
int idx2 = it2 - v.begin(); // idx2 = 7 (position after last 5)

// Count occurrences of 5
int count = upper_bound(v.begin(), v.end(), 5) - lower_bound(v.begin(), v.end(), 5);

// equal_range: returns pair of (lower_bound, upper_bound)
auto [lo, hi] = equal_range(v.begin(), v.end(), 5);
```

Patterns

Pattern 1: Lower/upper bound on answer

cpp

```
// "Find minimum k such that condition(k) is true"
// Condition must be monotone: false, false, ..., true, true, true
int lo = 1, hi = 1e9;
while(lo < hi){
    int mid = lo + (hi - lo) / 2;
    if(condition(mid)) hi = mid; // could be answer, search left
    else lo = mid + 1;          // not answer, search right
}
// lo is the answer
```

Pattern 2: "Maximize the minimum"

cpp

```
// Binary search on the answer; check if it's feasible
// Example: Given n stations, place k gas stations, maximize minimum distance
auto check = [&](double minDist) -> bool {
    int stations = 0;
    for(int i = 1; i < n; i++){
        stations += (int)((positions[i]-positions[i-1]) / minDist);
    }
    return stations <= k;
};
double lo = 0, hi = 1e9;
for(int iter = 0; iter < 100; iter++){
    double mid = (lo + hi) / 2;
    if(check(mid)) lo = mid;
    else hi = mid;
}
```

Pattern 3: Find first bad version (classic)

cpp

```
// Given sorted array, find first position where condition becomes true
int lo = 0, hi = n - 1, ans = -1;
while(lo <= hi){
    int mid = lo + (hi - lo) / 2;
    if(isBad(mid)){
        ans = mid;
        hi = mid - 1; // search left for earlier bad
    } else {
        lo = mid + 1;
    }
}
```

Common Mistakes

- Not sorting before binary search
- Integer overflow in $\text{mid} = (\text{lo} + \text{hi}) / 2$
- Off-by-one errors (using $<$ vs $\leq$, $\text{mid}-1$ vs mid)
- Using binary search on unsorted/non-monotone sequence
- Forgetting to handle the case where value isn't found

Practice Problems

- CSES: Apartments
 - CSES: Factory Machines
 - Leetcode 704: Binary Search
 - Leetcode 34: Find First and Last Position
 - Leetcode 875: Koko Eating Bananas (binary search on answer)
 - Codeforces 1201C: Maximum Median
-

7. PREFIX SUMS

What It Is

Precompute cumulative sums so that any subarray sum can be computed in $O(1)$ instead of $O(n)$.

Why It Matters

Prefix sums are a core building block for range queries. Without them, computing subarray sums requires $O(n)$ per query. With them, it's $O(1)$.

When to Use

- Sum of any subarray in $O(1)$ after $O(n)$ preprocessing
- Count of elements satisfying some condition in a range
- 2D problems requiring sum of rectangular subgrid

Time Complexity

- Preprocessing: $O(n)$
- Query: $O(1)$

Core Idea

$$\text{prefix}[i] = \text{arr}[0] + \text{arr}[1] + \dots + \text{arr}[i-1] \quad (\text{prefix}[0] = 0)$$

Sum of $\text{arr}[1..r] = \text{prefix}[r+1] - \text{prefix}[1]$

Example Code

```
cpp

int n;
cin >> n;
vector<int> arr(n);
for(auto &x : arr) cin >> x;

// Build prefix sum (1-indexed is cleaner)
vector<long long> prefix(n+1, 0);
for(int i = 0; i < n; i++)
    prefix[i+1] = prefix[i] + arr[i];

// Query: sum of arr[1..r] (0-indexed)
auto rangeSum = [&](int l, int r) -> long long {
    return prefix[r+1] - prefix[l];
};

// Example queries
cout << rangeSum(0, 4) << "\n";    // sum of arr[0..4]
cout << rangeSum(2, 7) << "\n";    // sum of arr[2..7]
```

2D Prefix Sums

```
cpp
```



```

int n, m;
cin >> n >> m;
vector<vector<int>> grid(n+1, vector<int>(m+1, 0));
for(int i = 1; i <= n; i++)
    for(int j = 1; j <= m; j++)
        cin >> grid[i][j];

// Build 2D prefix sum
vector<vector<long long>> P(n+1, vector<long long>(m+1, 0));
for(int i = 1; i <= n; i++)
    for(int j = 1; j <= m; j++)
        P[i][j] = grid[i][j] + P[i-1][j] + P[i][j-1] - P[i-1][j-1];

// Sum of rectangle (r1,c1) to (r2,c2) - 1-indexed
auto rectSum = [&](int r1, int c1, int r2, int c2) -> long long {
    return P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1];
};

```

Pattern: Count subarrays with given sum

```

cpp

// Count subarrays with sum = k
// Key insight: prefix[j] - prefix[i] = k means prefix[i] = prefix[j] - k
int countSubarraysWithSum(vector<int>& arr, int k){
    unordered_map<long long, int> freq;
    freq[0] = 1; // empty prefix
    long long sum = 0;
    int count = 0;
    for(int x : arr){
        sum += x;
        count += freq[sum - k];
        freq[sum]++;
    }
    return count;
}

```

Pattern: XOR prefix sums

```

cpp

```

```
// Same idea but with XOR instead of addition
// XOR of arr[l..r] = xorPrefix[r+1] ^ xorPrefix[l]
vector<int> xorPrefix(n+1, 0);
for(int i = 0; i < n; i++)
    xorPrefix[i+1] = xorPrefix[i] ^ arr[i];
```

Common Mistakes

- Using `int` instead of `long long` when values can be large
- Off-by-one errors: `prefix[r+1] - prefix[l]` for 0-indexed `arr[l..r]`
- Forgetting `prefix[0] = 0` (the base case)

Practice Problems

- CSES: Subarray Sums I & II
 - CSES: Subarray Divisibility
 - Leetcode 560: Subarray Sum Equals K
 - Leetcode 1314: Matrix Block Sum
-

8. FREQUENCY ARRAYS

What It Is

An array where index represents value and value represents how many times that index appeared in the input.

Why It Matters

Frequency arrays let you count, find, and group elements in $O(n)$ time. Essential for problems involving character counts, value distributions, and range queries.

When to Use

- Counting occurrences of values in a small range
- Finding the most/least frequent element
- Checking if two arrays have the same elements
- Finding elements appearing exactly k times

Time Complexity

- Build: $O(n)$
- Query: $O(1)$
- Space: $O(\text{max_value})$

Example Code

```
cpp

// Count character frequencies
string s;
cin >> s;
int freq[26] = {0};
for(char c : s) freq[c - 'a']++;

// Most frequent character
char mostFreq = 'a';
for(int i = 1; i < 26; i++)
    if(freq[i] > freq[mostFreq - 'a']) mostFreq = 'a' + i;

// All characters that appear at least k times
int k = 3;
for(int i = 0; i < 26; i++)
    if(freq[i] >= k) cout << (char)('a' + i) << "\n";
```

```
cpp

// Frequency array for integers in range [0, MAXVAL)
const int MAXVAL = 1e6 + 5;
int freq[MAXVAL] = {0};
int n; cin >> n;
for(int i = 0; i < n; i++){
    int x; cin >> x;
    freq[x]++;
}

// Count elements in range [l, r]
// First build prefix sum of freq array!
vector<int> prefFreq(MAXVAL+1, 0);
for(int i = 0; i < MAXVAL; i++)
    prefFreq[i+1] = prefFreq[i] + freq[i];

auto countInRange = [&](int l, int r) -> int {
    return prefFreq[r+1] - prefFreq[l];
};
```

Pattern: Mode, missing elements

```
cpp
```

```
// Find missing number in [1..n]
vector<int> freq(n+1, 0);
for(int x : arr) freq[x]++;
for(int i = 1; i <= n; i++)
    if(freq[i] == 0) cout << i << " is missing\n";

// Find the only number that appears once (all others appear twice)
int unique = 0;
for(int x : arr) unique ^= x; // XOR trick - faster than freq array
```

Common Mistakes

- Not allocating enough space (max value + 1)
- Using frequency arrays when values can be negative or very large (use hash map instead)
- Not resetting the array between test cases

Practice Problems

- Leetcode 448: Find All Numbers Disappeared in an Array
 - Leetcode 169: Majority Element
 - CSES: Missing Number
 - Codeforces 1328A: Divisibility Problem
-

9. TWO POINTERS

What It Is

A technique using two indices (pointers) that move through an array, typically from both ends toward the center, or both moving in the same direction.

Why It Matters

Converts many $O(n^2)$ brute-force solutions into $O(n)$ solutions.

When to Use

- Finding pairs with a given sum in a sorted array
- Removing duplicates or elements matching a condition
- Comparing characters in a string from both ends (palindrome)
- Merging two sorted arrays

Time Complexity

- $O(n)$ — each pointer moves at most n steps total

Types

Type 1: Opposite ends (sorted array)

```
cpp

// Find if any pair sums to target
bool twoSum(vector<int>& arr, int target){
    sort(arr.begin(), arr.end());
    int l = 0, r = arr.size() - 1;
    while(l < r){
        int sum = arr[l] + arr[r];
        if(sum == target) return true;
        else if(sum < target) l++;
        else r--;
    }
    return false;
}
```

Type 2: Same direction (both move right)

```
cpp

// Remove duplicates from sorted array in-place
int removeDuplicates(vector<int>& arr){
    int write = 0; // slow pointer (write position)
    for(int read = 0; read < arr.size(); read++){ // fast pointer
        if(read == 0 || arr[read] != arr[read-1])
            arr[write++] = arr[read];
    }
    return write; // new length
}
```

Type 3: Three pointers (3Sum)

```
cpp
```

```
// Find all triplets that sum to zero
vector<vector<int>> threeSum(vector<int>& arr){
    sort(arr.begin(), arr.end());
    vector<vector<int>> result;
    int n = arr.size();
    for(int i = 0; i < n-2; i++){
        if(i > 0 && arr[i] == arr[i-1]) continue; // skip duplicates
        int l = i+1, r = n-1;
        while(l < r){
            int sum = arr[i] + arr[l] + arr[r];
            if(sum == 0){
                result.push_back({arr[i], arr[l], arr[r]});
                while(l < r && arr[l] == arr[l+1]) l++; // skip dups
                while(l < r && arr[r] == arr[r-1]) r--; // skip dups
                l++; r--;
            } else if(sum < 0) l++;
            else r--;
        }
    }
    return result;
}
```

Common Patterns

Pattern: Count pairs with sum <= target

```
cpp

sort(arr.begin(), arr.end());
int l = 0, r = n - 1, count = 0;
while(l < r){
    if(arr[l] + arr[r] <= target){
        count += r - l; // all pairs (l, l+1), (l, l+2), ..., (l, r)
        l++;
    } else {
        r--;
    }
}
```

Pattern: Container with most water

```
cpp
```

```

int maxWater(vector<int>& height){
    int l = 0, r = height.size()-1, maxArea = 0;
    while(l < r){
        int area = min(height[l], height[r]) * (r - l);
        maxArea = max(maxArea, area);
        if(height[l] < height[r]) l++;
        else r--;
    }
    return maxArea;
}

```

Common Mistakes

- Forgetting to sort the array first (for opposite-ends approach)
- Moving wrong pointer — always move pointer with the smaller value (for sum problems)
- Not handling duplicates when required
- Using two pointers when the array isn't sorted and doesn't have a monotone property

Practice Problems

- Leetcode 167: Two Sum II - Input Array is Sorted
- Leetcode 11: Container With Most Water
- Leetcode 15: 3Sum
- CSES: Sum of Two Values
- Codeforces 6C: Alice, Bob and Chocolate

10. SLIDING WINDOW

What It Is

A technique for problems involving contiguous subarrays/substrings of a fixed or variable size. Maintain a "window" and slide it through the array.

Why It Matters

Converts $O(n*k)$ brute force to $O(n)$. Essential for maximum/minimum subarray problems.

When to Use

- Maximum sum subarray of size k (fixed window)
- Longest substring with at most k distinct characters (variable window)
- Minimum window substring
- Any "contiguous subarray/substring" problem

Time Complexity

- $O(n)$ — each element enters and exits the window at most once

Fixed-Size Window

```
cpp

// Maximum sum of any subarray of size k
int maxSumFixed(vector<int>& arr, int k){
    int n = arr.size();
    int windowSum = 0;
    for(int i = 0; i < k; i++) windowSum += arr[i]; // first window

    int maxSum = windowSum;
    for(int i = k; i < n; i++){
        windowSum += arr[i]; // add new element
        windowSum -= arr[i-k]; // remove old element
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}
```

Variable-Size Window (Expand/Contract)

```
cpp

// Longest subarray with sum <= target
int longestSubarrayWithSumAtMostK(vector<int>& arr, int target){
    int l = 0, sum = 0, maxLen = 0;
    for(int r = 0; r < arr.size(); r++){
        sum += arr[r]; // expand right
        while(sum > target) { // contract left while invalid
            sum -= arr[l];
            l++;
        }
        maxLen = max(maxLen, r - l + 1);
    }
    return maxLen;
}
```

```
cpp
```



```

// Minimum window containing all elements of another array
// (minimum window substring variant)
int minWindow(vector<int>& arr, int target){
    unordered_map<int, int> need, have;
    for(int x : target_arr) need[x]++;

    int formed = 0, required = need.size();
    int l = 0, minLen = INT_MAX;

    for(int r = 0; r < arr.size(); r++){
        have[arr[r]]++;
        if(need.count(arr[r]) && have[arr[r]] == need[arr[r]])
            formed++;

        while(formed == required){
            minLen = min(minLen, r - l + 1);
            have[arr[l]]--;
            if(need.count(arr[l]) && have[arr[l]] < need[arr[l]])
                formed--;
            l++;
        }
    }
    return minLen == INT_MAX ? -1 : minLen;
}

```

Pattern: Longest substring with at most k distinct chars

```

cpp

int longestKDistinct(string s, int k){
    unordered_map<char, int> count;
    int l = 0, maxLen = 0;
    for(int r = 0; r < s.size(); r++){
        count[s[r]]++;
        while(count.size() > k){
            count[s[l]]--;
            if(count[s[l]] == 0) count.erase(s[l]);
            l++;
        }
        maxLen = max(maxLen, r - l + 1);
    }
    return maxLen;
}

```

Pattern: Fixed window with multiset for max/min

```
cpp

// Maximum element in each window of size k (using multiset)
vector<int> maxInWindows(vector<int>& arr, int k){
    multiset<int, greater<int>> window;
    vector<int> result;
    for(int i = 0; i < arr.size(); i++){
        window.insert(arr[i]);
        if(i >= k) window.erase(window.find(arr[i-k]));
        if(i >= k-1) result.push_back(*window.begin());
    }
    return result;
    // Alternatively, use deque for O(n) instead of O(n log k)
}
```

Common Mistakes

- Forgetting to update window when shrinking (e.g., updating map/count)
- Using for the wrong problem type (only works for contiguous subarrays)
- Not handling edge cases (empty array, $k > n$)

Practice Problems

- CSES: Sum of Three Values
- Leetcode 3: Longest Substring Without Repeating Characters
- Leetcode 76: Minimum Window Substring
- Leetcode 239: Sliding Window Maximum
- Codeforces 380C: Sereja and Brackets

11. HASH MAPS

What It Is

A data structure that stores key-value pairs with $O(1)$ average time for insert, lookup, and delete. Implemented as `unordered_map` in C++ STL.

Why It Matters

Hash maps solve "does X exist?" and "how many times did X appear?" in $O(1)$. They turn $O(n^2)$ pair searches into $O(n)$.

When to Use

- Counting frequency of elements
- Checking membership
- Caching/memoization
- Two-sum type problems (find complement)
- Grouping elements by a key

Time Complexity

Operation	Average	Worst
Insert	$O(1)$	$O(n)$
Lookup	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

Worst case (hash collisions) is rarely triggered. Use `map` ($O(\log n)$) if you need guaranteed performance.

Key STL Functions

```
cpp
```

```

unordered_map<int, int> mp;
map<int, int> ordered_mp;    // sorted by key, O(log n)

// Insert / Update
mp[key] = value;
mp[key]++;                  // increment (default initializes to 0)
mp.insert({key, value});

// Lookup
mp[key]                    // creates if doesn't exist! (dangerous)
mp.at(key)                 // throws exception if not found
mp.count(key)              // 1 if exists, 0 if not
mp.find(key)               // returns iterator; == mp.end() if not found

// Iterate
for(auto& [key, val] : mp){
    cout << key << " " << val << "\n";
}

// Erase
mp.erase(key);
mp.erase(mp.find(key));

// Size
mp.size();
mp.empty();
mp.clear();

```

Example: Two Sum

```

cpp

vector<int> twoSum(vector<int>& nums, int target){
    unordered_map<int, int> seen; // value -> index
    for(int i = 0; i < nums.size(); i++){
        int complement = target - nums[i];
        if(seen.count(complement))
            return {seen[complement], i};
        seen[nums[i]] = i;
    }
    return {};
}

```

Example: Count subarrays with sum k (using prefix sums + hashmap)

```

cpp

```

```
int subarraySum(vector<int>& nums, int k){
    unordered_map<long long, int> freq;
    freq[0] = 1;
    long long sum = 0;
    int count = 0;
    for(int x : nums){
        sum += x;
        count += freq[sum - k];
        freq[sum]++;
    }
    return count;
}
```

map vs unordered_map

Feature	map	unordered_map
Order	Sorted by key	No order
Time (avg)	$O(\log n)$	$O(1)$
Time (worst)	$O(\log n)$	$O(n)$
Keys	Any comparable type	Hashable types
Use when	Need order/range queries	Need fast lookup

Common Mistakes

- Using `mp[key]` to check existence — this creates a default value if key doesn't exist. Use `mp.count(key)` or `mp.find(key)`
- Hash collisions can cause TLE with adversarial inputs on Codeforces — use custom hash or `map` if needed
- Not clearing between test cases

Custom Hash (anti-hack on Codeforces)

```
cpp
```

```

struct custom_hash {
    size_t operator()(int x) const {
        x = ((x >> 16) ^ x) * 0x45d9f3b;
        x = ((x >> 16) ^ x) * 0x45d9f3b;
        x = (x >> 16) ^ x;
        return x;
    }
};

unordered_map<int, int, custom_hash> mp;

```

Practice Problems

- Leetcode 1: Two Sum
- Leetcode 49: Group Anagrams
- Leetcode 560: Subarray Sum Equals K
- CSES: Sum of Two Values
- Codeforces 158B: Taxi

12. SETS

What It Is

A container that stores **unique** elements. `set` (ordered, $O(\log n)$), `unordered_set` (unordered, $O(1)$ avg).

Why It Matters

Sets give you fast membership testing, sorted iteration, and automatic deduplication.

When to Use

- Checking if an element exists
- Storing unique elements
- Finding smallest/largest element in a dynamic collection
- Finding the next/previous element (ordered set)

Time Complexity

Operation	set	unordered_set
Insert	$O(\log n)$	$O(1)$ avg
Erase	$O(\log n)$	$O(1)$ avg

Operation	set	unordered_set
Find	$O(\log n)$	$O(1)$ avg
Lower/upper bound	$O(\log n)$	N/A

Key STL Functions

```

cpp

set<int> s;
unordered_set<int> us;

// Insert (ignored if already exists)
s.insert(5);
s.insert({1, 2, 3, 4, 5}); // insert multiple

// Check existence
s.count(5)           // 1 if exists, 0 if not
s.find(5)            // iterator; == s.end() if not found
s.contains(5)        // C++20

// Erase
s.erase(5);
s.erase(s.find(5)); // erase by iterator

// Size
s.size();
s.empty();

// Ordered operations (set only, not unordered_set)
*s.begin()           // smallest element
*s.rbegin()          // largest element
*s.lower_bound(5)    // first element >= 5
*s.upper_bound(5)    // first element > 5
auto it = s.lower_bound(5);
if(it != s.begin()) --it; // previous element before 5

// Iterate (in sorted order for set)
for(int x : s) cout << x << " ";

```

multiset — allows duplicates

```

cpp

```

```

multiset<int> ms;
ms.insert(5);
ms.insert(5); // now has two 5s
ms.count(5); // returns 2
ms.erase(ms.find(5)); // remove ONE occurrence of 5
ms.erase(5); // remove ALL occurrences of 5

```

Example: Sliding Window Minimum using multiset

```

cpp
// Minimum element in window of size k
vector<int> windowMin(vector<int>& arr, int k){
    multiset<int> window;
    vector<int> result;
    for(int i = 0; i < arr.size(); i++){
        window.insert(arr[i]);
        if((int)window.size() > k)
            window.erase(window.find(arr[i-k]));
        if(i >= k-1)
            result.push_back(*window.begin());
    }
    return result;
}

```

Example: Finding closest value in set

```

cpp
set<int> s = {1, 3, 7, 11, 15};
int target = 8;

auto it = s.lower_bound(target); // first element >= 8, points to 11
int closest = *it; // 11

if(it != s.begin()){
    --it; // points to 7
    if(abs(*it - target) < abs(closest - target))
        closest = *it;
}

// closest = 7 (closer to 8 than 11)

```

Common Mistakes

- Using `s[0]` — sets don't support index access

- Erasing by value when multiset has duplicates — `s.erase(val)` removes ALL; use `s.erase(s.find(val))` for one
- Iterating and erasing simultaneously without proper iterator management

Practice Problems

- CSES: Distinct Numbers
 - CSES: Concert Tickets
 - Leetcode 220: Contains Duplicate III
 - Codeforces 61E: Enemy is a Troll
-

13. STACKS

What It Is

A Last-In-First-Out (LIFO) data structure. Think of a stack of plates — you add and remove from the top.

Why It Matters

Stacks model recursive calls, expression evaluation, and any problem where you need to "remember" previous states and process them in reverse order.

When to Use

- Matching brackets/parentheses
- Next greater/smaller element
- Depth-first search (iterative)
- Expression evaluation and parsing
- Monotonic stack problems

Time Complexity

All operations are $O(1)$.

Key STL Functions

```
cpp
```

```
stack<int> st;

st.push(5);    // add to top
st.pop();      // remove from top (doesn't return value!)
st.top();      // view top element
st.empty();    // true if empty
st.size();     // number of elements
```

Example: Matching Parentheses

```
cpp

bool isValid(string s){
    stack<char> st;
    for(char c : s){
        if(c == '(' || c == '[' || c == '{')
            st.push(c);
        else {
            if(st.empty()) return false;
            char top = st.top(); st.pop();
            if(c == ')' && top != '(') return false;
            if(c == ']' && top != '[') return false;
            if(c == '}' && top != '{') return false;
        }
    }
    return st.empty();
}
```

Monotonic Stack (Critical Pattern)

A monotonic stack maintains elements in increasing or decreasing order. Used to find next/previous greater/smaller elements.

```
cpp
```

```
// Next Greater Element: for each element, find the first element to its right
vector<int> nextGreater(vector<int>& arr){
    int n = arr.size();
    vector<int> result(n, -1);
    stack<int> st; // stores indices

    for(int i = 0; i < n; i++){
        while(!st.empty() && arr[st.top()] < arr[i]){
            result[st.top()] = arr[i]; // arr[i] is the next greater for st.top()
            st.pop();
        }
        st.push(i);
    }
    return result;
}
// Example: [2,1,3,4,1] -> [3,3,4,-1,-1]
```

cpp

```
// Largest Rectangle in Histogram (classic monotonic stack)
int largestRectangle(vector<int>& heights){
    stack<int> st;
    int maxArea = 0;
    heights.push_back(0); // sentinel

    for(int i = 0; i < heights.size(); i++){
        while(!st.empty() && heights[st.top()] > heights[i]){
            int h = heights[st.top()]; st.pop();
            int w = st.empty() ? i : i - st.top() - 1;
            maxArea = max(maxArea, h * w);
        }
        st.push(i);
    }
    return maxArea;
}
```

Common Mistakes

- Calling `st.pop()` on an empty stack — always check `st.empty()` first
- Forgetting that `st.pop()` doesn't return the value; use `st.top()` then `st.pop()`
- Not using indices in the stack when you need position information

Practice Problems

- Leetcode 20: Valid Parentheses

- Leetcode 739: Daily Temperatures (next greater element)
 - Leetcode 84: Largest Rectangle in Histogram
 - CSES: Nearest Smaller Values
 - Codeforces 797C: Minimal String
-

14. QUEUES

What It Is

A First-In-First-Out (FIFO) data structure. Think of a line at a store — first person in, first person out.

Why It Matters

Queues are the backbone of BFS (breadth-first search), level-order tree traversal, and any problem requiring processing in arrival order.

When to Use

- BFS (breadth-first search) — always
- Level-order tree traversal
- Simulating any "waiting line" process
- Task scheduling

Time Complexity

All operations $O(1)$.

Key STL Functions

```
cpp
queue<int> q;

q.push(5);      // add to back
q.pop();        // remove from front
q.front();      // view front element
q.back();       // view back element
q.empty();      // true if empty
q.size();       // number of elements
```

Example: BFS using Queue

```
cpp
```

```
// BFS shortest path in unweighted graph
vector<int> bfsDistance(int start, vector<vector<int>>& adj, int n){
    vector<int> dist(n, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);

    while(!q.empty()){
        int node = q.front(); q.pop();
        for(int neighbor : adj[node]){
            if(dist[neighbor] == -1){
                dist[neighbor] = dist[node] + 1;
                q.push(neighbor);
            }
        }
    }
    return dist;
}
```

Priority Queue (Heap Queue)

```
cpp

// Max-heap by default
priority_queue<int> pq;
pq.push(5);
pq.top();    // max element
pq.pop();    // remove max

// Min-heap
priority_queue<int, vector<int>, greater<int>> minpq;

// Custom comparator
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq
// Gives smallest pair first (by first element, then second)
```

Common Mistakes

- Using `queue` when you need LIFO (use `stack`) or priority ordering (use `priority_queue`)
- Calling `q.front()` or `q.pop()` on empty queue
- Forgetting that `priority_queue` is a max-heap by default

Practice Problems

- CSES: Message Route (BFS)
 - Leetcode 102: Binary Tree Level Order Traversal
 - Leetcode 1091: Shortest Path in Binary Matrix
 - Codeforces 580C: Kefa and Park
-

15. DEQUES

What It Is

Double-ended queue — supports $O(1)$ push/pop at both front and back. More powerful than both `stack` and `queue`.

Why It Matters

Dequeues enable the sliding window maximum/minimum algorithm in $O(n)$ — much better than the $O(n \log n)$ multiset approach.

When to Use

- Sliding window maximum/minimum (monotonic deque)
- When you need stack + queue functionality
- Palindrome checking (pop from both ends)
- BFS with 0-1 weights (0-1 BFS)

Key STL Functions

```
cpp
deque<int> dq;

dq.push_back(5); // add to back
dq.push_front(1); // add to front
dq.pop_back(); // remove from back
dq.pop_front(); // remove from front
dq.front(); // view front
dq.back(); // view back
dq.empty();
dq.size();
dq[i]; // random access by index
```

Monotonic Deque: Sliding Window Maximum in $O(n)$

```
cpp
```

```

vector<int> maxSlidingWindow(vector<int>& arr, int k){
    deque<int> dq; // stores indices, front = index of max element
    vector<int> result;

    for(int i = 0; i < arr.size(); i++){
        // Remove indices outside window
        while(!dq.empty() && dq.front() <= i-k)
            dq.pop_front();

        // Remove smaller elements from back (they'll never be max)
        while(!dq.empty() && arr[dq.back()] <= arr[i])
            dq.pop_back();

        dq.push_back(i);

        if(i >= k-1) // window is full
            result.push_back(arr[dq.front()]);
    }
    return result;
}

```

0-1 BFS with Deque

```

cpp

// BFS where edges have weight 0 or 1
// Use deque: push to front for 0-weight edges, push to back for 1-weight edges
vector<int> zeroOneBFS(int start, vector<vector<pair<int,int>>>& adj, int n){
    vector<int> dist(n, INT_MAX);
    deque<int> dq;
    dist[start] = 0;
    dq.push_back(start);

    while(!dq.empty()){
        int u = dq.front(); dq.pop_front();
        for(auto [v, w] : adj[u]){
            if(dist[u] + w < dist[v]){
                dist[v] = dist[u] + w;
                if(w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    return dist;
}

```

Practice Problems

- Leetcode 239: Sliding Window Maximum
 - Leetcode 862: Shortest Subarray with Sum at Least K
 - CSES: Maximum Subarray Sum II
-

16. LINKED LISTS

What It Is

A linear data structure where each node contains a value and a pointer to the next node.
Not contiguous in memory.

Why It Matters

Linked list questions are very common in interviews. They test pointer manipulation skills. In competitive programming they appear less often, but understanding them is crucial.

When to Use

- $O(1)$ insertion/deletion at known position
- When you don't know size in advance and size varies a lot
- Implementing other structures (stacks, queues, adjacency lists)

Time Complexity

Operation	Singly Linked	Doubly Linked
Access by index	$O(n)$	$O(n)$
Insert at head	$O(1)$	$O(1)$
Insert at tail	$O(n)$ / $O(1)$ with tail ptr	$O(1)$
Delete at head	$O(1)$	$O(1)$
Delete at tail	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$

Implementation

```
cpp
```



```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

// Create list: 1 -> 2 -> 3
ListNode* head = new ListNode(1);
head->next = new ListNode(2);
head->next->next = new ListNode(3);

// Traverse
ListNode* curr = head;
while(curr != nullptr){
    cout << curr->val << " ";
    curr = curr->next;
}

// Insert at beginning
ListNode* newHead = new ListNode(0);
newHead->next = head;
head = newHead;

// Delete a node (given node, not index)
// Trick: copy next node's value here, delete next node
void deleteNode(ListNode* node){
    node->val = node->next->val;
    ListNode* temp = node->next;
    node->next = node->next->next;
    delete temp;
}

```

Essential Patterns

Pattern 1: Reverse a linked list

cpp

```

ListNode* reverseList(ListNode* head){
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while(curr != nullptr){
        ListNode* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

```

Pattern 2: Detect cycle (Floyd's algorithm)

```

cpp

bool hasCycle(ListNode* head){
    ListNode *slow = head, *fast = head;
    while(fast && fast->next){
        slow = slow->next;
        fast = fast->next->next;
        if(slow == fast) return true;
    }
    return false;
}

```

Pattern 3: Find middle node

```

cpp

ListNode* findMiddle(ListNode* head){
    ListNode *slow = head, *fast = head;
    while(fast && fast->next){
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow; // slow is now at middle
}

```

Pattern 4: Merge two sorted lists

```

cpp

```

```

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2){
    ListNode dummy(0);
    ListNode* curr = &dummy;
    while(l1 && l2){
        if(l1->val <= l2->val){ curr->next = l1; l1 = l1->next; }
        else { curr->next = l2; l2 = l2->next; }
        curr = curr->next;
    }
    curr->next = l1 ? l1 : l2;
    return dummy.next;
}

```

Pattern 5: Find kth from end

cpp

```

ListNode* kthFromEnd(ListNode* head, int k){
    ListNode *fast = head, *slow = head;
    for(int i = 0; i < k; i++) fast = fast->next; // advance fast by k
    while(fast) { slow = slow->next; fast = fast->next; }
    return slow;
}

```

Common Mistakes

- Dereferencing null pointer (always check `(curr != nullptr)` before `(curr->next)`)
- Losing track of nodes during manipulation (save `(next)` before overwriting)
- Memory leaks from not `(delete)`ing nodes (less critical in CP but good habit)
- Not handling edge cases: empty list, single-element list

Practice Problems

- Leetcode 206: Reverse Linked List
- Leetcode 21: Merge Two Sorted Lists
- Leetcode 141: Linked List Cycle
- Leetcode 19: Remove Nth Node From End
- Leetcode 234: Palindrome Linked List

17. RECURSION

What It Is

A function that calls itself with a smaller version of the problem, until reaching a base case.

Why It Matters

Recursion is the foundation for DFS, tree traversal, backtracking, and dynamic programming. Many problems that look hard iteratively are elegant recursively.

When to Use

- Tree/graph traversal (DFS)
- Divide and conquer (merge sort, binary search)
- Backtracking (generate all possibilities)
- Problems with natural self-similar structure

Time Complexity

Depends on the recurrence. Use the **recurrence tree** or **Master theorem**.

Master theorem: If $T(n) = aT(n/b) + O(n^d)$:

- If $a < b^d$: $T(n) = O(n^d)$
- If $a = b^d$: $T(n) = O(n^d \cdot \log n)$
- If $a > b^d$: $T(n) = O(n^{\log_b(a)})$

Anatomy of Recursion

1. Base case: the simplest input where you know the answer directly
2. Recursive case: reduce the problem to a smaller version + combine

cpp

```

// Factorial:  $n! = n * (n-1)!$ 
int factorial(int n){
    if(n <= 1) return 1;          // base case
    return n * factorial(n-1);    // recursive case
}

// Fibonacci:  $fib(n) = fib(n-1) + fib(n-2)$ 
int fib(int n){
    if(n <= 1) return n;          // base cases: fib(0)=0, fib(1)=1
    return fib(n-1) + fib(n-2);  //  $O(2^n)$  – use memoization!
}

// Binary search (recursive)
int binSearch(vector<int>& arr, int lo, int hi, int target){
    if(lo > hi) return -1;        // base case: not found
    int mid = lo + (hi - lo) / 2;
    if(arr[mid] == target) return mid;
    if(arr[mid] < target) return binSearch(arr, mid+1, hi, target);
    return binSearch(arr, lo, mid-1, target);
}

```

Recursion Patterns

Pattern 1: Subsets (all 2^n)

```

cpp

void generateSubsets(vector<int>& arr, int idx, vector<int>& current, vector<ve
    if(idx == arr.size()){
        result.push_back(current);
        return;
    }
    // Choice 1: include arr[idx]
    current.push_back(arr[idx]);
    generateSubsets(arr, idx+1, current, result);
    current.pop_back();

    // Choice 2: exclude arr[idx]
    generateSubsets(arr, idx+1, current, result);
}

```

Pattern 2: Permutations (all $n!$)

```

cpp

```

```

void generatePermutations(vector<int>& arr, int start, vector<vector<int>>& res)
{
    if(start == arr.size()){
        result.push_back(arr);
        return;
    }
    for(int i = start; i < arr.size(); i++){
        swap(arr[start], arr[i]);
        generatePermutations(arr, start+1, result);
        swap(arr[start], arr[i]); // backtrack
    }
}

```

Pattern 3: Divide and conquer (merge sort)

```

cpp

void mergeSort(vector<int>& arr, int lo, int hi){
    if(lo >= hi) return;
    int mid = lo + (hi - lo) / 2;
    mergeSort(arr, lo, mid);
    mergeSort(arr, mid+1, hi);
    merge(arr, lo, mid, hi); // combine sorted halves
}

```

Stack Overflow Warning

Each recursive call uses stack space. For n up to 10^6 , iterative is safer.

```

cpp

// Increase stack size for deep recursion (Linux only, competitive programming)
// Or convert to iterative using explicit stack

```

Common Mistakes

- Forgetting the base case (infinite recursion → stack overflow)
- Base case that doesn't cover all terminal conditions
- Not returning the recursive result: `return recursiveCall(...)` not just `recursiveCall(...)`
- Exponential time due to recomputing the same subproblems (fix with memoization → DP)

Practice Problems

- Leetcode 78: Subsets
- Leetcode 46: Permutations

- Leetcode 104: Maximum Depth of Binary Tree
 - CSES: Apple Division
 - Codeforces 115A: Party (subset sum)
-

18. BACKTRACKING

What It Is

A systematic recursive technique to explore all possible solutions by building candidates incrementally and abandoning (backtracking) candidates that can't lead to valid solutions.

Why It Matters

Backtracking is the go-to approach for constraint satisfaction problems: N-queens, Sudoku, combinations, permutations with constraints.

When to Use

- Generate all valid combinations/permutations
- Constraint satisfaction (Sudoku, N-queens)
- Word search in a grid
- Any "find all solutions" problem

Time Complexity

Generally $O(\text{branching_factor}^{\text{depth}})$. Often exponential, but pruning can drastically reduce actual runtime.

The Backtracking Template

```
function backtrack(state):
    if state is a complete solution:
        add to results
        return

    for each choice:
        if choice is valid:
            make choice
            backtrack(new state)
            undo choice (backtrack)
```

Example: N-Queens

```
cpp
```

```

int n;
vector<vector<string>> result;

bool isSafe(vector<string>& board, int row, int col){
    // Check column above
    for(int i = 0; i < row; i++) if(board[i][col] == 'Q') return false;
    // Check upper-left diagonal
    for(int i=row-1, j=col-1; i>=0 && j>=0; i--,j--) if(board[i][j]=='Q') return false;
    // Check upper-right diagonal
    for(int i=row-1, j=col+1; i>=0 && j<n; i--,j++) if(board[i][j]=='Q') return false;
    return true;
}

void solve(vector<string>& board, int row){
    if(row == n){ result.push_back(board); return; }
    for(int col = 0; col < n; col++){
        if(isSafe(board, row, col)){
            board[row][col] = 'Q';    // make choice
            solve(board, row + 1);    // recurse
            board[row][col] = '.';    // undo choice
        }
    }
}

```

Example: Combination Sum

```

cpp

// Find all combinations that sum to target (can reuse elements)
void backtrack(vector<int>& candidates, int target, int start,
               vector<int>& current, vector<vector<int>>& result){
    if(target == 0){ result.push_back(current); return; }
    if(target < 0) return;    // pruning!

    for(int i = start; i < candidates.size(); i++){
        current.push_back(candidates[i]);
        backtrack(candidates, target - candidates[i], i, current, result);
        current.pop_back();
    }
}

```

Example: Word Search in Grid

```

cpp

```



```

bool dfs(vector<vector<char>>& board, string& word, int i, int j, int k){
    if(k == word.size()) return true;
    if(i<0 || i>=board.size() || j<0 || j>=board[0].size()) return false;
    if(board[i][j] != word[k]) return false;

    char temp = board[i][j];
    board[i][j] = '#'; // mark visited

    bool found = dfs(board, word, i+1, j, k+1) ||
                 dfs(board, word, i-1, j, k+1) ||
                 dfs(board, word, i, j+1, k+1) ||
                 dfs(board, word, i, j-1, k+1);

    board[i][j] = temp; // restore (backtrack)
    return found;
}

```

Pruning Strategies

1. **Early termination:** if current partial solution already violates constraints, stop
2. **Sort first:** sorting candidates lets you `break` early when value exceeds target
3. **Bitmask visited:** faster than `vector<bool> visited`

Common Mistakes

- Forgetting to undo changes (the "backtrack" step)
- Not pruning enough (solution TLEs)
- Modifying global state without reverting

Practice Problems

- Leetcode 39: Combination Sum
- Leetcode 51: N-Queens
- Leetcode 79: Word Search
- Leetcode 17: Letter Combinations of a Phone Number
- CSES: Chessboard and Queens

19. TREES

What It Is

A connected, acyclic graph with n nodes and $n-1$ edges. A rooted tree has one designated root node.

Why It Matters

Trees model hierarchical data. Binary trees, BSTs, heaps, segment trees — all are tree variants. Tree traversal patterns appear in almost every hard problem.

Key Terminology

- **Root:** top node, no parent
- **Leaf:** node with no children
- **Height:** longest root-to-leaf path
- **Depth:** distance from root
- **Parent/Child/Sibling:** relative positions
- **Subtree:** a node and all its descendants

Binary Tree Implementation

```
cpp

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

Traversals

```
cpp
```

```

// Inorder: Left → Root → Right (gives sorted order for BST)
void inorder(TreeNode* root){
    if(!root) return;
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}

// Preorder: Root → Left → Right (used to copy tree)
void preorder(TreeNode* root){
    if(!root) return;
    cout << root->val << " ";
    preorder(root->left);
    preorder(root->right);
}

// Postorder: Left → Right → Root (used to delete tree)
void postorder(TreeNode* root){
    if(!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->val << " ";
}

// Level-order (BFS): level by level
void levelOrder(TreeNode* root){
    if(!root) return;
    queue<TreeNode*> q;
    q.push(root);
    while(!q.empty()){
        TreeNode* node = q.front(); q.pop();
        cout << node->val << " ";
        if(node->left) q.push(node->left);
        if(node->right) q.push(node->right);
    }
}

```

Essential Tree Problems

cpp

```

// Height of tree
int height(TreeNode* root){
    if(!root) return 0;
    return 1 + max(height(root->left), height(root->right));
}

// Count nodes
int countNodes(TreeNode* root){
    if(!root) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}

// Check if balanced (left and right heights differ by at most 1)
int checkBalanced(TreeNode* root){
    if(!root) return 0;
    int left = checkBalanced(root->left);
    if(left == -1) return -1;
    int right = checkBalanced(root->right);
    if(right == -1) return -1;
    if(abs(left - right) > 1) return -1;
    return 1 + max(left, right);
}

bool isBalanced(TreeNode* root){ return checkBalanced(root) != -1; }

// Lowest Common Ancestor
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q){
    if(!root || root == p || root == q) return root;
    TreeNode* left = lca(root->left, p, q);
    TreeNode* right = lca(root->right, p, q);
    if(left && right) return root; // p and q in different subtrees
    return left ? left : right;
}

// Diameter of tree (longest path between any two nodes)
int diameter = 0;
int dfs(TreeNode* node){
    if(!node) return 0;
    int l = dfs(node->left), r = dfs(node->right);
    diameter = max(diameter, l + r);
    return 1 + max(l, r);
}

// Path sum: is there a root-to-leaf path with given sum?
bool hasPathSum(TreeNode* root, int sum){
    if(!root) return false;
    if(!root->left && !root->right) return root->val == sum;
}

```

```

    return hasPathSum(root->left, sum - root->val) ||
           hasPathSum(root->right, sum - root->val);
}

```

N-ary Tree (General Tree, used in CP)

```

cpp

// Represented as adjacency list
vector<vector<int>> adj(n+1);
// or with parent array
vector<int> parent(n+1);

```

Tree on Graphs (CP Style)

```

cpp

// DFS to compute subtree sizes
int subtreeSize[MAXN];
void dfs(int u, int par, vector<vector<int>>& adj){
    subtreeSize[u] = 1;
    for(int v : adj[u]){
        if(v == par) continue;
        dfs(v, u, adj);
        subtreeSize[u] += subtreeSize[v];
    }
}

```

Common Mistakes

- Not handling null pointers (check `if(!root) return ...`)
- Confusing preorder/inorder/postorder
- Forgetting to pass parent in undirected graph DFS to avoid going back

Practice Problems

- Leetcode 104: Maximum Depth of Binary Tree
 - Leetcode 226: Invert Binary Tree
 - Leetcode 112: Path Sum
 - Leetcode 236: LCA of Binary Tree
 - CSES: Tree Diameter
 - CSES: Tree Distances I
-

20. BINARY SEARCH TREES (BST)

What It Is

A binary tree where for every node: all left subtree values < node value, and all right subtree values > node value.

Why It Matters

BSTs give $O(\log n)$ average search, insert, and delete. In competitive programming, `std::set` and `std::map` are self-balancing BSTs ($O(\log n)$ guaranteed).

Key Property

Inorder traversal of a BST gives sorted order.

Time Complexity

Operation	Average (balanced)	Worst (skewed)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

Implementation

```
cpp
```

```

struct BST {
    TreeNode* root = nullptr;

    void insert(int val){
        root = insert(root, val);
    }

    TreeNode* insert(TreeNode* node, int val){
        if(!node) return new TreeNode(val);
        if(val < node->val) node->left = insert(node->left, val);
        else if(val > node->val) node->right = insert(node->right, val);
        return node; // duplicate: do nothing
    }

    bool search(int val){
        TreeNode* curr = root;
        while(curr){
            if(val == curr->val) return true;
            curr = val < curr->val ? curr->left : curr->right;
        }
        return false;
    }

    // Find minimum (leftmost node)
    int findMin(){
        TreeNode* curr = root;
        while(curr->left) curr = curr->left;
        return curr->val;
    }
};

```

Validate BST

```

cpp

bool isValidBST(TreeNode* root, long long lo = LLONG_MIN, long long hi = LLONG_
    if(!root) return true;
    if(root->val <= lo || root->val >= hi) return false;
    return isValidBST(root->left, lo, root->val) &&
        isValidBST(root->right, root->val, hi);
}

```

Kth Smallest in BST (inorder)

```

cpp

```

```
int kthSmallest(TreeNode* root, int& k){
    if(!root) return -1;
    int left = kthSmallest(root->left, k);
    if(k == 0) return left;
    if(--k == 0) return root->val;
    return kthSmallest(root->right, k);
}
```

In Competitive Programming

Use `std::set` / `std::map` instead of implementing BST manually.

```
cpp
set<int> s;           // self-balancing BST (Red-Black Tree)
map<int,int> m;       // BST of key-value pairs
```

Common Mistakes

- Not using `LLONG_MIN` / `LLONG_MAX` as bounds when validating (root value might be `INT_MIN`)
- Confusing BST with a sorted array (BST supports dynamic operations)

Practice Problems

- Leetcode 98: Validate Binary Search Tree
- Leetcode 230: Kth Smallest Element in BST
- Leetcode 235: LCA in BST
- CSES: Hotel Queries (use BST / multiset)

21. HEAPS / PRIORITY QUEUES

What It Is

A complete binary tree where each parent is larger (max-heap) or smaller (min-heap) than its children. Gives $O(\log n)$ insert and $O(1)$ peek at max/min.

Why It Matters

Priority queues are essential for Dijkstra's, Prim's, top-k elements, and scheduling problems.

When to Use

- Finding k-th largest/smallest elements efficiently

- Dijkstra's shortest path algorithm
- Merge k sorted lists
- Scheduling (always process the most urgent task next)
- Median of a data stream

Time Complexity

Operation	Time
Push	$O(\log n)$
Pop	$O(\log n)$
Top (peek)	$O(1)$
Build heap from n elements	$O(n)$

STL Priority Queue

```
cpp

// Max-heap (default)
priority_queue<int> pq;
pq.push(5); pq.push(3); pq.push(8);
pq.top(); // 8
pq.pop(); // removes 8

// Min-heap
priority_queue<int, vector<int>, greater<int>> minpq;

// Max-heap of pairs (sorted by first element, then second)
priority_queue<pair<int,int>> pq2;
pq2.push({3, 1}); pq2.push({1, 5}); pq2.push({3, 2});
// top = {3, 2} (largest first, then largest second)

// Custom comparator
struct Compare {
    bool operator()(pair<int,int> a, pair<int,int> b){
        return a.second > b.second; // min-heap by second element
    }
};
priority_queue<pair<int,int>, vector<pair<int,int>>, Compare> customPQ;
```

Example: K Largest Elements

```
cpp
```

```

vector<int> kLargest(vector<int>& arr, int k){
    priority_queue<int, vector<int>, greater<int>> minpq; // min-heap of size
    for(int x : arr){
        minpq.push(x);
        if(minpq.size() > k) minpq.pop(); // remove smallest
    }
    vector<int> result;
    while(!minpq.empty()){ result.push_back(minpq.top()); minpq.pop(); }
    return result;
    // The min-heap keeps track of the k largest seen so far
}

```

Example: Merge K Sorted Lists

```

cpp

ListNode* mergeKLists(vector<ListNode*>& lists){
    // Min-heap: (value, list_index, node_pointer)
    priority_queue<pair<int,ListNode*>, vector<pair<int,ListNode*>>,
        greater<pair<int,ListNode*>>> pq;

    for(auto head : lists)
        if(head) pq.push({head->val, head});

    ListNode dummy(0);
    ListNode* curr = &dummy;
    while(!pq.empty()){
        auto [val, node] = pq.top(); pq.pop();
        curr->next = node;
        curr = curr->next;
        if(node->next) pq.push({node->next->val, node->next});
    }
    return dummy.next;
}

```

Example: Median of Data Stream

```

cpp

```

```
// Two heaps: max-heap for lower half, min-heap for upper half
priority_queue<int> lower;           // max-heap
priority_queue<int, vector<int>, greater<int>> upper; // min-heap

void addNum(int num){
    lower.push(num);
    upper.push(lower.top()); lower.pop(); // balance
    if(upper.size() > lower.size()){
        lower.push(upper.top()); upper.pop();
    }
}

double findMedian(){
    if(lower.size() > upper.size()) return lower.top();
    return (lower.top() + upper.top()) / 2.0;
}
```

Common Mistakes

- Using max-heap when you need min-heap (negate values or use `greater<>()`)
- Accessing `pq.top()` on empty heap
- Forgetting that `priority_queue` doesn't support iteration or access by index

Practice Problems

- Leetcode 215: Kth Largest Element in an Array
- Leetcode 295: Find Median from Data Stream
- CSES: Hospital (Greedy + heap)
- CSES: Josephus Problem II
- Codeforces 1041D: Glider

22. GRAPH FUNDAMENTALS

What It Is

A graph is a collection of nodes (vertices) and connections (edges). Used to model networks, maps, dependencies, and relationships.

Key Terminology

- **Directed/Undirected:** edges have direction or not
- **Weighted/Unweighted:** edges have costs or not
- **Connected:** path exists between any two nodes

- **Cyclic/Acyclic:** contains cycles or not
- **Degree:** number of edges connected to a node

Representations

1. Adjacency List (most common in CP)

```
cpp

int n, m; // n nodes, m edges
vector<vector<int>> adj(n+1); // 1-indexed

// Undirected edge u-v
adj[u].push_back(v);
adj[v].push_back(u);

// Directed edge u->v
adj[u].push_back(v);

// Weighted edge u-v with weight w
vector<vector<pair<int,int>>> adj(n+1); // adj[u] = {v, weight}
adj[u].push_back({v, w});
adj[v].push_back({u, w});
```

2. Adjacency Matrix (for dense graphs)

```
cpp

int mat[MAXN][MAXN]; // mat[u][v] = weight (0 if no edge)
mat[u][v] = w;
mat[v][u] = w; // if undirected
```

3. Edge List (for Kruskal's MST)

```
cpp

vector<tuple<int,int,int>> edges; // {weight, u, v}
edges.push_back({w, u, v});
sort(edges.begin(), edges.end()); // sort by weight
```

Standard Input Reading for Graphs

```
cpp
```

```

int n, m;
cin >> n >> m;
vector<vector<int>> adj(n+1);

for(int i = 0; i < m; i++){
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u); // remove for directed graph
}

```

Graph Properties to Check

Connected components:

```

cpp

vector<bool> visited(n+1, false);
int components = 0;
for(int i = 1; i <= n; i++){
    if(!visited[i]){
        dfs(i, adj, visited);
        components++;
    }
}

```

Bipartite check:

```

cpp

```

```
// A graph is bipartite if it can be 2-colored (no odd cycles)
vector<int> color(n+1, -1);
bool isBipartite = true;

function<void(int, int)> bfs = [&](int start, int col){
    queue<int> q;
    q.push(start);
    color[start] = col;
    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : adj[u]){
            if(color[v] == -1){ color[v] = 1-color[u]; q.push(v); }
            else if(color[v] == color[u]) isBipartite = false;
        }
    }
};
```

Space and Time Complexity of Representations

	Adjacency List	Adjacency Matrix
Space	$O(V + E)$	$O(V^2)$
Add edge	$O(1)$	$O(1)$
Check edge	$O(\text{degree})$	$O(1)$
Iterate neighbors	$O(\text{degree})$	$O(V)$
Best for	Sparse graphs	Dense graphs

Practice Problems

- CSES: Building Roads
- CSES: Counting Rooms
- Leetcode 547: Number of Provinces

23. DEPTH-FIRST SEARCH (DFS)

What It Is

A graph traversal that explores as deep as possible before backtracking. Uses a stack (recursive call stack).

Why It Matters

DFS is the basis for cycle detection, topological sort, connected components, path finding, and many tree algorithms.

When to Use

- Detecting cycles
- Finding connected components
- Topological ordering
- Finding paths (not necessarily shortest)
- Flood fill

Time Complexity

- $O(V + E)$ where V = vertices, E = edges

Implementation

Recursive DFS:

```
cpp

void dfs(int u, vector<vector<int>>& adj, vector<bool>& visited){
    visited[u] = true;
    // Process node u here
    cout << u << " ";

    for(int v : adj[u]){
        if(!visited[v]) dfs(v, adj, visited);
    }
}

// Call:
vector<bool> visited(n+1, false);
dfs(1, adj, visited);
```

Iterative DFS (avoids stack overflow for large n):

```
cpp
```

```

void iterativeDFS(int start, vector<vector<int>>& adj, int n){
    vector<bool> visited(n+1, false);
    stack<int> st;
    st.push(start);

    while(!st.empty()){
        int u = st.top(); st.pop();
        if(visited[u]) continue;
        visited[u] = true;
        cout << u << " ";
        for(int v : adj[u]) if(!visited[v]) st.push(v);
    }
}

```

DFS Patterns

Cycle detection (undirected graph):

```

cpp

bool hasCycle(int u, int parent, vector<vector<int>>& adj, vector<bool>& visited)
{
    visited[u] = true;
    for(int v : adj[u]){
        if(!visited[v]){
            if(hasCycle(v, u, adj, visited)) return true;
        } else if(v != parent) return true; // back edge = cycle
    }
    return false;
}

```

Cycle detection (directed graph):

```

cpp

// Need three states: unvisited(0), in-stack(1), done(2)
vector<int> state(n+1, 0);

bool hasCycleDirected(int u, vector<vector<int>>& adj){
    state[u] = 1; // in-stack
    for(int v : adj[u]){
        if(state[v] == 1) return true; // back edge
        if(state[v] == 0 && hasCycleDirected(v, adj)) return true;
    }
    state[u] = 2; // done
    return false;
}

```


DFS on grid (flood fill):

```
cpp

int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

void floodFill(vector<vector<int>>& grid, int r, int c, int newColor, int oldCo
    if(r < 0 || r >= grid.size() || c < 0 || c >= grid[0].size()) return;
    if(grid[r][c] != oldColor) return;
    grid[r][c] = newColor;
    for(int d = 0; d < 4; d++)
        floodFill(grid, r+dx[d], c+dy[d], newColor, oldColor);
}
```

DFS to compute entry/exit times (useful for many advanced algorithms):

```
cpp

int timer = 0;
vector<int> tin(n+1), tout(n+1);

void dfs(int u, int par){
    tin[u] = timer++;
    for(int v : adj[u]) if(v != par) dfs(v, u);
    tout[u] = timer++;
}

// Node v is ancestor of u if tin[v] <= tin[u] && tout[u] <= tout[v]
```

Common Mistakes

- Visiting nodes multiple times without a `visited` array
- Stack overflow for large graphs (use iterative or increase stack size)
- Treating directed graph as undirected (not passing parent for cycle detection)

Practice Problems

- CSES: Counting Rooms
 - CSES: Labyrinth
 - Leetcode 200: Number of Islands
 - Leetcode 695: Max Area of Island
 - Codeforces 580D: Kefa and Dishes
-

24. BREADTH-FIRST SEARCH (BFS)

What It Is

A graph traversal that explores all nodes at the current level before moving to the next level. Uses a queue.

Why It Matters

BFS finds the **shortest path** in an unweighted graph. It's the go-to algorithm for minimum steps, minimum cost (unweighted), and level-based problems.

When to Use

- Shortest path in unweighted graph
- Minimum number of moves/steps
- Level-order processing
- Finding all nodes within k distance

Time Complexity

- $O(V + E)$

BFS Template

```
cpp

void bfs(int start, vector<vector<int>>& adj, int n){
    vector<int> dist(n+1, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);

    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : adj[u]){
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

Multi-Source BFS

```
cpp
```

```

// BFS from multiple sources simultaneously
// Example: Find distance from any '1' to nearest '0' in grid
void multiSourceBFS(vector<vector<int>>& grid){
    int rows = grid.size(), cols = grid[0].size();
    vector<vector<int>> dist(rows, vector<int>(cols, -1));
    queue<pair<int,int>> q;

    // Initialize all sources
    for(int i = 0; i < rows; i++)
        for(int j = 0; j < cols; j++)
            if(grid[i][j] == 1){ dist[i][j] = 0; q.push({i,j}); }

    int dx[] = {0,0,1,-1}, dy[] = {1,-1,0,0};
    while(!q.empty()){
        auto [x, y] = q.front(); q.pop();
        for(int d = 0; d < 4; d++){
            int nx = x+dx[d], ny = y+dy[d];
            if(nx>=0 && nx<rows && ny>=0 && ny<cols && dist[nx][ny]==-1){
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }
}

```

BFS on Grid

cpp

```

// Shortest path in maze (0=open, 1=wall)
int shortestPath(vector<vector<int>>& maze, pair<int,int> start, pair<int,int>
    int rows = maze.size(), cols = maze[0].size();
    vector<vector<int>> dist(rows, vector<int>(cols, -1));
    queue<pair<int,int>> q;

    dist[start.first][start.second] = 0;
    q.push(start);

    int dx[] = {0,0,1,-1}, dy[] = {1,-1,0,0};
    while(!q.empty()){
        auto [x, y] = q.front(); q.pop();
        if(make_pair(x,y) == end) return dist[x][y];
        for(int d = 0; d < 4; d++){
            int nx = x+dx[d], ny = y+dy[d];
            if(nx>=0 && nx<rows && ny>=0 && ny<cols
                && maze[nx][ny]==0 && dist[nx][ny]==-1){
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }
    return -1; // not reachable
}

```

BFS vs DFS: When to Use Which?

BFS	DFS
Shortest path (unweighted)	Cycle detection
Level-order traversal	Topological sort
All nodes at distance k	Connected components
When solution is shallow	When solution is deep

Common Mistakes

- Using BFS for shortest path in WEIGHTED graphs (use Dijkstra instead)
- Not marking visited before pushing to queue (causes duplicate processing)
- Using the wrong data structure (stack instead of queue)

Practice Problems

- CSES: Message Route
 - CSES: Monsters (multi-source BFS)
 - Leetcode 1091: Shortest Path in Binary Matrix
 - Leetcode 994: Rotting Oranges (multi-source BFS)
 - Codeforces 540C: Ice Cave
-

25. TOPOLOGICAL SORT

What It Is

An ordering of nodes in a directed acyclic graph (DAG) such that for every directed edge $u \rightarrow v$, u appears before v .

Why It Matters

Topological sort handles dependency ordering: compile order, course prerequisites, build systems.

When to Use

- Course prerequisite problems ("can I finish all courses?")
- Task scheduling with dependencies
- Any problem on a DAG where order matters
- Detecting cycles in directed graphs

Two Algorithms

Kahn's Algorithm (BFS-based):

```
cpp
```

```

vector<int> topoSort(int n, vector<vector<int>>& adj){
    vector<int> indegree(n+1, 0);
    for(int u = 1; u <= n; u++)
        for(int v : adj[u]) indegree[v]++;

    queue<int> q;
    for(int i = 1; i <= n; i++)
        if(indegree[i] == 0) q.push(i); // nodes with no dependencies

    vector<int> order;
    while(!q.empty()){
        int u = q.front(); q.pop();
        order.push_back(u);
        for(int v : adj[u]){
            indegree[v]--;
            if(indegree[v] == 0) q.push(v);
        }
    }

    if(order.size() != n) return {}; // cycle detected
    return order;
}

```

DFS-based (post-order reverse):

```

cpp

vector<int> order;
vector<bool> visited(n+1, false);

void dfs(int u, vector<vector<int>>& adj){
    visited[u] = true;
    for(int v : adj[u]) if(!visited[v]) dfs(v, adj);
    order.push_back(u); // add AFTER processing all neighbors
}

// Call dfs for each unvisited node, then reverse order
for(int i = 1; i <= n; i++) if(!visited[i]) dfs(i, adj);
reverse(order.begin(), order.end());

```

Example: Course Schedule

```

cpp

```

```

bool canFinish(int n, vector<vector<int>>& prerequisites){
    vector<vector<int>> adj(n);
    vector<int> indegree(n, 0);

    for(auto& p : prerequisites){
        adj[p[1]].push_back(p[0]);
        indegree[p[0]]++;
    }

    queue<int> q;
    for(int i = 0; i < n; i++) if(indegree[i] == 0) q.push(i);

    int count = 0;
    while(!q.empty()){
        int u = q.front(); q.pop();
        count++;
        for(int v : adj[u]) if(--indegree[v] == 0) q.push(v);
    }

    return count == n; // if all courses processed, no cycle
}

```

Common Mistakes

- Applying topological sort to an undirected graph
- Not checking for cycles (if not all nodes in result, there's a cycle)
- Using wrong edge direction

Practice Problems

- Leetcode 207: Course Schedule
- Leetcode 210: Course Schedule II
- CSES: Course Schedule
- CSES: Longest Flight Route (DP on DAG)
- Codeforces 510C: Fox And Names

26. DISJOINT SET UNION (UNION-FIND)

What It Is

A data structure that tracks which elements belong to the same group (component). Supports two operations: find (which group?) and union (merge groups).

Why It Matters

DSU detects when adding an edge creates a cycle, and can count connected components in $O(\alpha(n)) \approx O(1)$ amortized.

When to Use

- Kruskal's MST algorithm
- Detecting cycles while adding edges
- Counting connected components
- "Are X and Y connected?" queries

Time Complexity

- $O(\alpha(n))$ per operation (nearly $O(1)$) with path compression + union by rank
- α is the inverse Ackermann function, essentially constant

Implementation

```
cpp
```



```

struct DSU {
    vector<int> parent, rank;
    int components;

    DSU(int n) : parent(n+1), rank(n+1, 0), components(n) {
        iota(parent.begin(), parent.end(), 0); // parent[i] = i
    }

    int find(int x){
        if(parent[x] != x)
            parent[x] = find(parent[x]); // path compression
        return parent[x];
    }

    bool unite(int x, int y){
        x = find(x); y = find(y);
        if(x == y) return false; // already same component
        if(rank[x] < rank[y]) swap(x, y);
        parent[y] = x; // union by rank
        if(rank[x] == rank[y]) rank[x]++;
        components--;
        return true;
    }

    bool connected(int x, int y){
        return find(x) == find(y);
    }
};

// Usage
DSU dsu(n);
dsu.unite(1, 2);
dsu.unite(2, 3);
dsu.connected(1, 3); // true
dsu.components;      // number of connected components

```

Example: Cycle Detection with DSU

cpp

```
bool hasCycle(int n, vector<pair<int,int>>& edges){
    DSU dsu(n);
    for(auto [u, v] : edges){
        if(dsu.connected(u, v)) return true; // adding this edge creates cycle
        dsu.unite(u, v);
    }
    return false;
}
```

Example: Kruskal's with DSU

```
cpp
// See Minimum Spanning Trees section
```

DSU with Rollback (for offline algorithms)

```
cpp
// When you need to undo union operations
// Use union by rank only (NO path compression – path compression is non-undoable)
stack<pair<int*,int>> history; // pointer to parent, old value

void unite(int x, int y){
    x = find(x); y = find(y);
    if(x == y) return;
    if(rank[x] < rank[y]) swap(x, y);
    history.push({&parent[y], parent[y]});
    history.push({&rank[x], rank[x]});
    parent[y] = x;
    if(rank[x] == rank[y]) rank[x]++;
}

void rollback(){
    auto [ptr, val] = history.top(); history.pop();
    *ptr = val;
    auto [ptr2, val2] = history.top(); history.pop();
    *ptr2 = val2;
}
```

Common Mistakes

- Forgetting path compression (makes it $O(\log n)$ instead of $O(\alpha(n))$)
- Not using union by rank (can degenerate to $O(n)$)
- Indexing errors (0-indexed vs 1-indexed)

Practice Problems

- CSES: Road Construction
 - CSES: Building Teams
 - Leetcode 684: Redundant Connection
 - Codeforces 25D: Roads not only in Berland
-

27. GREEDY ALGORITHMS

What It Is

Making the locally optimal choice at each step, hoping it leads to a globally optimal solution.

Why It Matters

Greedy algorithms are simple and efficient when they work. The challenge is proving they work (or recognizing when they don't).

When to Use

- Activity selection / interval scheduling
- Huffman encoding
- Minimum coin change (with standard coin sets)
- Fractional knapsack
- Many graph problems (Dijkstra, Prim, Kruskal)

How to Recognize a Greedy Problem

1. You make a sequence of choices
2. Each choice is local and irrevocable
3. The optimal substructure holds (optimal global = optimal local choices)

Key Greedy Patterns

Pattern 1: Interval Scheduling (maximize non-overlapping intervals)

```
cpp
```

```
// Sort by end time, greedily pick earliest-ending non-overlapping interval
int maxNonOverlapping(vector<pair<int,int>>& intervals){
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b){
        return a.second < b.second; // sort by end time
    });
    int count = 0, lastEnd = INT_MIN;
    for(auto& [start, end] : intervals){
        if(start >= lastEnd){
            count++;
            lastEnd = end;
        }
    }
    return count;
}
```

Pattern 2: Minimum platforms / Meeting Rooms

```
cpp

// Minimum rooms needed to hold all meetings
int minRooms(vector<pair<int,int>>& meetings){
    vector<int> starts, ends;
    for(auto& [s, e] : meetings){ starts.push_back(s); ends.push_back(e); }
    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());

    int rooms = 0, maxRooms = 0, j = 0;
    for(int i = 0; i < meetings.size(); i++){
        if(starts[i] < ends[j]) rooms++; // new meeting starts before one ends
        else j++; // a meeting ended
        maxRooms = max(maxRooms, rooms);
    }
    return maxRooms;
}
```

Pattern 3: Fractional Knapsack

```
cpp
```

```
// Maximize value in knapsack – can take fractions of items
double fractionalKnapsack(vector<pair<int,int>>& items, int capacity){
    // Sort by value/weight ratio descending
    sort(items.begin(), items.end(), [](auto& a, auto& b){
        return (double)a.first/a.second > (double)b.first/b.second;
    });
    double totalValue = 0;
    for(auto& [value, weight] : items){
        if(capacity >= weight){
            totalValue += value;
            capacity -= weight;
        } else {
            totalValue += (double)value * capacity / weight;
            break;
        }
    }
    return totalValue;
}
```

Pattern 4: Assign tasks to minimize total cost

cpp

```
// Match smallest tasks with most capable workers
// Sort both, pair smallest with smallest (or largest with largest)
sort(tasks.begin(), tasks.end());
sort(workers.begin(), workers.end());
```

Pattern 5: Gas Station

cpp

```
// Can you complete the circular route?
int canCompleteCircuit(vector<int>& gas, vector<int>& cost){
    int total = 0, tank = 0, start = 0;
    for(int i = 0; i < gas.size(); i++){
        total += gas[i] - cost[i];
        tank += gas[i] - cost[i];
        if(tank < 0){ start = i+1; tank = 0; }
    }
    return total >= 0 ? start : -1;
}
```

Common Mistakes

- Assuming greedy works without proof (e.g., 0/1 knapsack requires DP, not greedy)

- Wrong sorting criterion
- Not handling ties correctly

Practice Problems

- CSES: Movie Festival (interval scheduling)
 - CSES: Stick Lengths
 - Leetcode 435: Non-overlapping Intervals
 - Leetcode 134: Gas Station
 - Codeforces 545C: Woodcutters
-

28. BIT MANIPULATION

What It Is

Operating directly on individual bits of integers. Very fast (single CPU instruction).

Why It Matters

Bit tricks can solve subset problems, check divisibility, speed up DP, and compress boolean arrays.

Bit Operators

&	AND	$5 \& 3 = 1$	$(101 \& 011 = 001)$
	OR	$5 3 = 7$	$(101 011 = 111)$
^	XOR	$5 \wedge 3 = 6$	$(101 \wedge 011 = 110)$
~	NOT	$\sim 5 = -6$	(flips all bits)
<<	Left shift	$5 \ll 1 = 10$	(multiply by 2)
>>	Right shift	$5 \gg 1 = 2$	(divide by 2)

Essential Bit Tricks

```
cpp
```

```

// Check if bit i is set
bool isSet(int n, int i) { return (n >> i) & 1; }

// Set bit i
int setBit(int n, int i) { return n | (1 << i); }

// Clear bit i
int clearBit(int n, int i) { return n & ~(1 << i); }

// Toggle bit i
int toggleBit(int n, int i) { return n ^ (1 << i); }

// Check if n is power of 2
bool isPowerOf2(int n) { return n > 0 && (n & (n-1)) == 0; }

// Count set bits
int popcount(int n) { return __builtin_popcount(n); } // fast built-in
int popcountLL(long long n) { return __builtin_popcountll(n); }

// Lowest set bit (used in Fenwick tree)
int lsb(int n) { return n & (-n); }

// Remove lowest set bit
int removeLSB(int n) { return n & (n-1); }

// Get highest set bit position
int highBit(int n) { return 31 - __builtin_clz(n); } // for n > 0

// XOR: a^b^b = a (cancellation property)
// XOR: find the only number that appears once
int findUnique(vector<int>& arr){
    int result = 0;
    for(int x : arr) result ^= x;
    return result;
}

```

Bitmask Enumeration (Subsets)

cpp

```

int n = 4; // set of 4 elements
// Enumerate all 2^n subsets
for(int mask = 0; mask < (1 << n); mask++){
    // Each bit of mask represents inclusion of that element
    for(int i = 0; i < n; i++){
        if(mask & (1 << i)) cout << i << " "; // element i is in subset
    }
    cout << "\n";
}

// Enumerate all subsets of a given mask
for(int sub = mask; sub > 0; sub = (sub-1) & mask){
    // process sub
}

```

DP with Bitmask (Bitmask DP)

```

cpp

// Travelling Salesman Problem (TSP) - O(n^2 * 2^n)
int n;
vector<vector<int>> dist;
int dp[1<<16][16]; // dp[mask][last] = min cost to visit cities in mask, ending at last

int tsp(){
    memset(dp, 0x3f, sizeof(dp));
    dp[1][0] = 0; // start at city 0

    for(int mask = 1; mask < (1<<n); mask++){
        for(int u = 0; u < n; u++){
            if(!(mask & (1<<u))) continue;
            if(dp[mask][u] == 0x3f3f3f3f) continue;
            for(int v = 0; v < n; v++){
                if(mask & (1<<v)) continue;
                int newMask = mask | (1<<v);
                dp[newMask][v] = min(dp[newMask][v], dp[mask][u] + dist[u][v]);
            }
        }
    }

    int ans = INT_MAX;
    int fullMask = (1<<n) - 1;
    for(int u = 1; u < n; u++) ans = min(ans, dp[fullMask][u] + dist[u][0]);
    return ans;
}

```


Common Mistakes

- Integer overflow: `1 << 31` overflows int, use `1LL << 31`
- Operator precedence: `n & 1 == 0` is `n & (1 == 0)` — add parentheses: `(n & 1) == 0`
- Signed vs unsigned behavior of right shift for negative numbers

Practice Problems

- Leetcode 191: Number of 1 Bits
 - Leetcode 231: Power of Two
 - Leetcode 136: Single Number
 - CSES: Apple Division (bitmask brute force)
 - Codeforces 864B: Polycarp and Letters
-

29. DYNAMIC PROGRAMMING

What It Is

Breaking a problem into overlapping subproblems, solving each once, and storing results to avoid recomputation.

Why It Matters

DP solves optimization problems (min/max), counting problems, and feasibility problems that would otherwise require exponential time.

When to Use

- Problem has **optimal substructure** (optimal solution uses optimal sub-solutions)
- Problem has **overlapping subproblems** (same sub-problems are solved multiple times)
- Asking for minimum, maximum, count, or feasibility of all combinations

The Two Approaches

Top-down (Memoization): Recursive + cache results

```
cpp
```

```
map<int, long long> memo;
long long fib(int n){
    if(n <= 1) return n;
    if(memo.count(n)) return memo[n]; // cached
    return memo[n] = fib(n-1) + fib(n-2);
}
```

Bottom-up (Tabulation): Fill a table iteratively

```
cpp
vector<long long> dp(n+1);
dp[0] = 0; dp[1] = 1;
for(int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
```

Classic DP Problems

1. 0/1 Knapsack

```
cpp
// n items, each with weight w[i] and value v[i]. Capacity W.
// dp[j] = max value achievable with capacity j
vector<int> dp(W+1, 0);
for(int i = 0; i < n; i++){
    for(int j = W; j >= w[i]; j--){ // iterate backward!
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}
// Answer: dp[W]
```

2. Unbounded Knapsack (can use items multiple times)

```
cpp
vector<int> dp(W+1, 0);
for(int j = 0; j <= W; j++){ // iterate forward!
    for(int i = 0; i < n; i++){
        if(w[i] <= j) dp[j] = max(dp[j], dp[j-w[i]] + v[i]);
    }
}
```

3. Longest Common Subsequence (LCS)

```
cpp
```

```
// dp[i][j] = LCS of a[0..i-1] and b[0..j-1]
int lcs(string a, string b){
    int m = a.size(), n = b.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for(int i = 1; i <= m; i++){
        for(int j = 1; j <= n; j++){
            if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}
```

4. Longest Increasing Subsequence (LIS)

```
cpp

// O(n log n) approach using patience sorting
int lis(vector<int>& arr){
    vector<int> tails; // tails[i] = smallest tail element of increasing subsequence of length i+1
    for(int x : arr){
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if(it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return tails.size();
}
```

5. Coin Change

```
cpp

// Minimum coins to make amount
// dp[j] = min coins to make sum j
vector<int> dp(amount+1, INT_MAX);
dp[0] = 0;
for(int j = 1; j <= amount; j++){
    for(int coin : coins){
        if(coin <= j && dp[j-coin] != INT_MAX)
            dp[j] = min(dp[j], dp[j-coin] + 1);
    }
}
```

6. Edit Distance

```
cpp
```

```
// dp[i][j] = min operations to convert a[0..i-1] to b[0..j-1]
int editDistance(string a, string b){
    int m=a.size(), n=b.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for(int i=0;i<=m;i++) dp[i][0]=i;
    for(int j=0;j<=n;j++) dp[0][j]=j;
    for(int i=1;i<=m;i++){
        for(int j=1;j<=n;j++){
            if(a[i-1]==b[j-1]) dp[i][j]=dp[i-1][j-1];
            else dp[i][j]=1+min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
        }
    }
    return dp[m][n];
}
```

7. Matrix Chain Multiplication

```
cpp
// dp[i][j] = min multiplications to compute A[i..j]
// dp[i][j] = min over k in [i,j-1] of dp[i][k] + dp[k+1][j] + dims[i]*dims[k+1]*dims[j]
// Interval DP pattern
for(int len = 2; len <= n; len++){
    for(int i = 0; i+len-1 < n; i++){
        int j = i+len-1;
        dp[i][j] = INT_MAX;
        for(int k = i; k < j; k++){
            dp[i][j] = min(dp[i][j], dp[i][k]+dp[k+1][j]+dims[i]*dims[k+1]*dims[j]);
        }
    }
}
```

DP on Strings

```
cpp
```

```
// Count distinct subsequences of s that equal t
// dp[i][j] = ways to form t[0..j-1] using s[0..i-1]
long long countSubseq(string s, string t){
    int m=s.size(), n=t.size();
    vector<vector<long long>> dp(m+1, vector<long long>(n+1, 0));
    for(int i=0;i<=m;i++) dp[i][0]=1; // empty t is always achievable
    for(int i=1;i<=m;i++){
        for(int j=1;j<=n;j++){
            dp[i][j]=dp[i-1][j]; // don't use s[i]
            if(s[i-1]==t[j-1]) dp[i][j]+=dp[i-1][j-1]; // use s[i]
        }
    }
    return dp[m][n];
}
```

DP on Trees

```
cpp

// dp[u][0] = max independent set not including u
// dp[u][1] = max independent set including u
void dfs(int u, int par){
    dp[u][1] = 1; // include u
    dp[u][0] = 0; // exclude u
    for(int v : adj[u]){
        if(v == par) continue;
        dfs(v, u);
        dp[u][1] += dp[v][0]; // if u is included, v must be excluded
        dp[u][0] += max(dp[v][0], dp[v][1]); // if u excluded, v can be either
    }
}
```

Identifying DP States

Ask: "What information do I need to make the next decision?"

- Index position $\rightarrow dp[i]$
- Two indices $\rightarrow dp[i][j]$
- Remaining capacity $\rightarrow dp[capacity]$
- Visited set $\rightarrow dp[bitmask]$
- Two endpoints of interval $\rightarrow dp[l][r]$

Common Mistakes

- Wrong base cases
- Off-by-one errors in indexing

- Using `int` when values overflow (use `long long`)
- Forgetting to handle the "impossible" state (use `INT_MAX/INF` but be careful with overflow when adding to it)
- Bottom-up iteration in wrong direction for knapsack

Practice Problems

- CSES: Dice Combinations
- CSES: Minimizing Coins
- CSES: Coin Combinations I & II
- CSES: Book Shop (0/1 Knapsack)
- CSES: Longest Increasing Subsequence
- Leetcode 300: Longest Increasing Subsequence
- Leetcode 72: Edit Distance
- Leetcode 1143: Longest Common Subsequence
- Codeforces 189A: Cut Ribbon

30. SHORTEST PATH ALGORITHMS

What It Is

Algorithms to find minimum-cost path between nodes in a weighted graph.

Which Algorithm?

Condition	Algorithm
Unweighted graph	BFS — $O(V+E)$
Non-negative weights	Dijkstra — $O((V+E) \log V)$
Negative weights, no negative cycles	Bellman-Ford — $O(VE)$
All pairs, dense graph	Floyd-Warshall — $O(V^3)$
DAG	DP + topological sort — $O(V+E)$
0-1 weights	0-1 BFS (deque) — $O(V+E)$

Dijkstra's Algorithm

```
cpp
```

```

// Single source shortest path, non-negative weights
// O((V+E) log V) using priority queue
vector<long long> dijkstra(int src, vector<vector<pair<int,int>>>& adj, int n){
    vector<long long> dist(n+1, LLONG_MAX);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>>

    dist[src] = 0;
    pq.push({0, src});

    while(!pq.empty()){
        auto [d, u] = pq.top(); pq.pop();

        if(d > dist[u]) continue; // outdated entry in PQ

        for(auto [v, w] : adj[u]){
            if(dist[u] + w < dist[v]){
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

```

Bellman-Ford

cpp

```
// Works with negative weights, detects negative cycles
// O(VE)
vector<long long> bellmanFord(int src, vector<tuple<int,int,int>>& edges, int n)
{
    vector<long long> dist(n+1, LLONG_MAX);
    dist[src] = 0;

    for(int i = 0; i < n-1; i++){ // relax all edges n-1 times
        for(auto [u, v, w] : edges){
            if(dist[u] != LLONG_MAX && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }

    // Check for negative cycles
    for(auto [u, v, w] : edges){
        if(dist[u] != LLONG_MAX && dist[u] + w < dist[v])
            return {}; // negative cycle exists
    }
    return dist;
}
```

Floyd-Warshall

```
cpp

// All pairs shortest path
// O(V^3), good for V <= 500
vector<vector<long long>> floydWarshall(int n, vector<vector<long long>>& dist)
{
    // Initialize: dist[i][i] = 0, dist[i][j] = edge weight or INF
    for(int k = 1; k <= n; k++)
        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= n; j++)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
    return dist;
}
```

Common Mistakes

- Using Dijkstra with negative edges (gives wrong answer)
- Integer overflow: distances can be up to $V \cdot \text{maxWeight}$, use `long long`
- Not skipping outdated PQ entries in Dijkstra
- Floyd-Warshall: visiting k in inner loops, not outer (k must be outermost)

Practice Problems

- CSES: Shortest Routes I (Dijkstra)

- CSES: Shortest Routes II (Floyd-Warshall)
 - CSES: High Score (Bellman-Ford with negative cycles)
 - Leetcode 743: Network Delay Time
 - Codeforces 20C: Dijkstra?
-

31. MINIMUM SPANNING TREES (MST)

What It Is

A spanning tree of a weighted graph with minimum total edge weight. A spanning tree connects all vertices with $V-1$ edges.

Two Algorithms

Kruskal's (sort edges, greedily add):

```
cpp
// O(E log E) - best for sparse graphs
long long kruskal(int n, vector<tuple<int,int,int>>& edges){
    sort(edges.begin(), edges.end()); // sort by weight
    DSU dsu(n);
    long long totalWeight = 0;
    int edgesUsed = 0;

    for(auto [w, u, v] : edges){
        if(dsu.unite(u, v)){
            totalWeight += w;
            edgesUsed++;
            if(edgesUsed == n-1) break;
        }
    }
    return (edgesUsed == n-1) ? totalWeight : -1; // -1 if not connected
}
```

Prim's (grow MST from a node using priority queue):

```
cpp
```

```
// O((V+E) log V) – better for dense graphs
long long prims(int n, vector<vector<pair<int,int>>>& adj){
    vector<long long> key(n+1, LLONG_MAX);
    vector<bool> inMST(n+1, false);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>>

    key[1] = 0;
    pq.push({0, 1});
    long long totalWeight = 0;

    while(!pq.empty()){
        auto [w, u] = pq.top(); pq.pop();
        if(inMST[u]) continue;
        inMST[u] = true;
        totalWeight += w;

        for(auto [v, cost] : adj[u]){
            if(!inMST[v] && cost < key[v]){
                key[v] = cost;
                pq.push({key[v], v});
            }
        }
    }
    return totalWeight;
}
```

Common Mistakes

- Applying MST to directed graphs (use different algorithm)
- Not checking if graph is connected

Practice Problems

- CSES: Road Reparation (MST)
- Leetcode 1584: Min Cost to Connect All Points
- Codeforces 1624E: Build a Fence (Kruskal)

32. SEGMENT TREES

What It Is

A binary tree data structure for efficiently answering range queries (sum, min, max) and supporting point or range updates.

Why It Matters

Segment trees do range queries in $O(\log n)$ with $O(\log n)$ updates, better than prefix sums (which are $O(1)$ query but $O(n)$ update).

When to Use

- Range sum/min/max queries with updates
- Range assignment (lazy propagation)
- Interval problems with dynamic changes

Time Complexity

Operation	Time
Build	$O(n)$
Point update	$O(\log n)$
Range query	$O(\log n)$
Range update (lazy)	$O(\log n)$

Implementation (Range Sum + Point Update)

```
cpp
```

```

struct SegTree {
    int n;
    vector<long long> tree;

    SegTree(int n) : n(n), tree(4*n, 0) {}

    void build(vector<int>& arr, int node, int lo, int hi){
        if(lo == hi){ tree[node] = arr[lo]; return; }
        int mid = lo + (hi-lo)/2;
        build(arr, 2*node, lo, mid);
        build(arr, 2*node+1, mid+1, hi);
        tree[node] = tree[2*node] + tree[2*node+1];
    }

    void update(int node, int lo, int hi, int idx, int val){
        if(lo == hi){ tree[node] = val; return; }
        int mid = lo + (hi-lo)/2;
        if(idx <= mid) update(2*node, lo, mid, idx, val);
        else update(2*node+1, mid+1, hi, idx, val);
        tree[node] = tree[2*node] + tree[2*node+1];
    }

    long long query(int node, int lo, int hi, int l, int r){
        if(r < lo || hi < l) return 0; // out of range
        if(l <= lo && hi <= r) return tree[node]; // fully inside
        int mid = lo + (hi-lo)/2;
        return query(2*node, lo, mid, l, r) + query(2*node+1, mid+1, hi, l, r);
    }

    // Public interface
    void build(vector<int>& arr){ build(arr, 1, 0, n-1); }
    void update(int idx, int val){ update(1, 0, n-1, idx, val); }
    long long query(int l, int r){ return query(1, 0, n-1, l, r); }
};

```

Lazy Propagation (Range Update)

cpp

```

struct LazySegTree {
    int n;
    vector<long long> tree, lazy;

    LazySegTree(int n) : n(n), tree(4*n, 0), lazy(4*n, 0) {}

    void push_down(int node, int lo, int hi){
        if(lazy[node]){
            int mid = lo + (hi-lo)/2;
            tree[2*node] += lazy[node] * (mid-lo+1);
            lazy[2*node] += lazy[node];
            tree[2*node+1] += lazy[node] * (hi-mid);
            lazy[2*node+1] += lazy[node];
            lazy[node] = 0;
        }
    }

    void update(int node, int lo, int hi, int l, int r, long long val){
        if(r < lo || hi < l) return;
        if(l <= lo && hi <= r){
            tree[node] += val * (hi-lo+1);
            lazy[node] += val;
            return;
        }
        push_down(node, lo, hi);
        int mid = lo + (hi-lo)/2;
        update(2*node, lo, mid, l, r, val);
        update(2*node+1, mid+1, hi, l, r, val);
        tree[node] = tree[2*node] + tree[2*node+1];
    }

    long long query(int node, int lo, int hi, int l, int r){
        if(r < lo || hi < l) return 0;
        if(l <= lo && hi <= r) return tree[node];
        push_down(node, lo, hi);
        int mid = lo + (hi-lo)/2;
        return query(2*node, lo, mid, l, r) + query(2*node+1, mid+1, hi, l, r);
    }
};

```

Common Mistakes

- Allocating tree of size $(2*n)$ instead of $(4*n)$ (need $4n$ for safety)
- Off-by-one in range boundaries
- Forgetting push_down in lazy segment tree before accessing children

Practice Problems

- CSES: Range Sum Queries I & II
 - CSES: Range Minimum Queries
 - CSES: Range Updates and Sums (lazy propagation)
 - Codeforces 339D: Xenia and Tree
-

33. SPARSE TABLES

What It Is

A static data structure for answering range minimum/maximum queries in $O(1)$ after $O(n \log n)$ preprocessing. Only works for static arrays (no updates).

When to Use

- Many range min/max queries on a fixed array
- RMQ (Range Minimum Query) problems
- LCA (Lowest Common Ancestor) using Euler tour

Time Complexity

Operation	Time
Build	$O(n \log n)$
Query (min/max)	$O(1)$
Query (sum, not idempotent)	$O(\log n)$

Implementation

```
cpp
```

```

struct SparseTable {
    int LOG = 20;
    vector<vector<int>> table;
    vector<int> log2;
    int n;

    SparseTable(vector<int>& arr){
        n = arr.size();
        table.assign(LOG, vector<int>(n));
        log2.assign(n+1, 0);
        for(int i = 2; i <= n; i++) log2[i] = log2[i/2] + 1;

        table[0] = arr; // base case: ranges of length 1

        for(int j = 1; j < LOG; j++)
            for(int i = 0; i + (1<<j) <= n; i++)
                table[j][i] = min(table[j-1][i], table[j-1][i + (1<<(j-1))]);
    }

    // O(1) RMQ – uses overlap (valid for idempotent ops like min/max)
    int query(int l, int r){
        int k = log2[r-l+1];
        return min(table[k][l], table[k][r - (1<<k) + 1]);
    }
};

```

Practice Problems

- CSES: Range Minimum Queries
- Leetcode 2334: Subarray With Elements Greater Than Varying Threshold

34. COMMON COMPETITIVE PROGRAMMING PATTERNS

Pattern 1: Read Until EOF

```

cpp

int x;
while(cin >> x){
    // process x
}

```

```
cpp

int T;
cin >> T;
while(T--){
    // solve one test case
}
```

Pattern 3: Output "YES"/"NO"

```
cpp

cout << (condition ? "YES" : "NO") << "\n";
```

Pattern 4: Coordinate Compression

```
cpp

// When values are large but count is small
// Map values to [0, k-1] where k = number of distinct values
vector<int> vals = arr; // copy
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());

auto compress = [&](int x) -> int {
    return lower_bound(vals.begin(), vals.end(), x) - vals.begin();
};
// Now compress(arr[i]) gives the compressed index [0, k-1]
```

Pattern 5: Offline Processing

```
cpp

// When queries can be answered out of order
// Group queries by some criterion, sort, then sweep
struct Query { int l, r, idx; };
vector<Query> queries;
// Sort queries by r (or l), process in sorted order
sort(queries.begin(), queries.end(), [](auto& a, auto& b){ return a.r < b.r; })
```

Pattern 6: Fast Power (Modular Exponentiation)

```
cpp
```



```

long long power(long long base, long long exp, long long mod){
    long long result = 1;
    base %= mod;
    while(exp > 0){
        if(exp & 1) result = result * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return result;
}
// Usage: power(2, 10, MOD) // 2^10 mod MOD

```

Pattern 7: Modular Arithmetic

```

cpp

const long long MOD = 1e9 + 7;
// Addition
(a + b) % MOD
// Multiplication – always mod after multiply
(a * b) % MOD
// Subtraction – add MOD before subtracting to avoid negatives
(a - b + MOD) % MOD
// Division (requires modular inverse – only when MOD is prime)
// a/b mod p = a * modInverse(b, p) mod p
long long modInverse(long long a, long long mod){ return power(a, mod-2, mod);

```

Pattern 8: Square Root Decomposition

```

cpp

// Split array into blocks of size sqrt(n)
// Good for range queries with updates when segment tree is overkill
int blockSize = sqrt(n);
vector<int> block(n/blockSize + 1, 0);
// block[i] = aggregate of arr[i*blockSize .. (i+1)*blockSize - 1]

```

Pattern 9: Binary Lifting (LCA)

```

cpp

```

```

// Find LCA in O(log n) after O(n log n) preprocessing
int LOG = 20;
int up[MAXN][LOG]; // up[v][j] = 2^j-th ancestor of v
int depth[MAXN];

void dfs(int v, int par, int d){
    up[v][0] = par;
    depth[v] = d;
    for(int j = 1; j < LOG; j++) up[v][j] = up[up[v][j-1]][j-1];
    for(int u : adj[v]) if(u != par) dfs(u, v, d+1);
}

int lca(int u, int v){
    if(depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];
    for(int j = 0; j < LOG; j++) if((diff >> j) & 1) u = up[u][j];
    if(u == v) return u;
    for(int j = LOG-1; j >= 0; j--)
        if(up[u][j] != up[v][j]){ u = up[u][j]; v = up[v][j]; }
    return up[u][0];
}

```

Pattern 10: Euler Tour / Flattening a Tree

```

cpp

// Convert tree to array for range queries on subtrees
int tin[MAXN], tout[MAXN], euler[MAXN], timer = 0;

void dfs(int u, int par){
    tin[u] = timer;
    euler[timer++] = u;
    for(int v : adj[u]) if(v != par) dfs(v, u);
    tout[u] = timer - 1;
}

// All nodes in subtree of u correspond to array indices [tin[u], tout[u]]

```

35. COMMON CSES PATTERNS

CSES (cses.fi) is a high-quality problem set organized by topic. Here are the key patterns for each section:

Introductory Problems

- Read input, simple simulation, math tricks
- Key: understand problem statement carefully, edge cases

Sorting and Searching

- Binary search on answer (Apartments, Factory Machines)
- Greedy with sorting (Movie Festival, Tasks and Deadlines)
- Two-pointer (Sum of Two Values)
- Key insight: "What is the minimum maximum?" → binary search on answer

Dynamic Programming

- Dice Combinations → unbounded knapsack variant
- Minimizing Coins → coin change DP
- Grid paths → 2D DP, $dp[i][j]$ = ways to reach cell (i,j)
- Edit distance, LCS → string DP

Graph Algorithms

- Connected components → DFS/BFS/DSU
- Shortest path → BFS (unweighted), Dijkstra (weighted)
- Tree queries → DFS ordering, LCA, binary lifting

Tree Algorithms

- Most problems use DFS + DP on tree
- Subtree queries → Euler tour + segment tree
- Path queries → LCA + binary lifting

Mathematics

- Counting paths modulo → DP with modular arithmetic
 - Primes → Sieve of Eratosthenes
 - GCD, LCM → Euclidean algorithm
-

36. COMMON CODEFORCES PATTERNS

By Rating

800-1000 (Div. 4 A-B, Div. 3 A-B):

- Simulation / implementation
- Simple math (divisibility, parity)
- Reading problem carefully — the trick is often obvious
- Frequency count
- Sorting + greedy

1100-1300 (Div. 3 C-D, Div. 4 C-E):

- Binary search on answer
- Prefix sums + pigeonhole
- Constructive algorithms
- Greedy with proof
- Simple DP

1400-1600 (Div. 2 B-C):

- Graph BFS/DFS
- Two pointers / sliding window
- Ad-hoc observations
- Bitmask DP (small n)
- String manipulation

1700-1900 (Div. 2 C-D):

- Segment trees / Fenwick trees
- DP optimizations (monotone queue, divide and conquer)
- Flow / matching (basic)
- Number theory (modular inverse, Euler's theorem)
- Sparse tables / binary lifting

2000+ (Div. 1 B-C):

- Complex DP transitions
- Advanced graph algorithms
- Offline algorithms
- Interactive problems
- Heavy-light decomposition

Strategy Tips

- Read all problems first — sometimes later problems are easier
- Start coding simple brute force, optimize if needed

- When stuck: think about what's special about the optimal answer
 - Common Codeforces tricks:
 - Odd/even parity arguments
 - Pigeonhole principle
 - Complementary counting
 - Think about the answer backwards
-

100 MOST IMPORTANT STL FUNCTIONS

I/O

cpp

```
1. ios_base::sync_with_stdio(false) // fast I/O
2. cin.tie(NULL)                    // fast I/O
3. cout << "\n"                     // faster than endl (no flush)
4. getline(cin, str)                // read full line
5. printf / scanf                    // C-style I/O (sometimes faster)
```

Containers

cpp

```

// vector
6.  v.push_back(x)           // 0(1) amortized
7.  v.pop_back()            // 0(1)
8.  v.size()                 // 0(1)
9.  v.empty()                // 0(1)
10. v.clear()                // 0(n)
11. v.resize(n, val)         // resize
12. v.reserve(n)             // pre-allocate
13. v.back() / v.front()     // last/first element
14. v.insert(it, x)          // 0(n)
15. v.erase(it)              // 0(n)
16. v.assign(n, val)         // set n elements to val

// string
17. s.substr(pos, len)       // substring
18. s.find(t)                 // first occurrence (string::npos if not found)
19. s.rfind(t)                // last occurrence
20. s.replace(pos, len, t)    // replace
21. to_string(n)              // int to string
22. stoi(s) / stoll(s)       // string to int/long long
23. s.push_back(c) / s.pop_back() // append/remove char
24. s += t                    // concatenate
25. isalpha/isdigit/islower/isupper/tolower/toupper // char functions

// set / unordered_set
26. s.insert(x)               // 0(log n) / 0(1)
27. s.erase(x)                // 0(log n) / 0(1)
28. s.count(x)                 // 1 or 0
29. s.find(x)                  // iterator
30. s.lower_bound(x)           // first >= x (set only)
31. s.upper_bound(x)           // first > x (set only)
32. *s.begin() / *s.rbegin()  // min/max (set only)

// map / unordered_map
33. m[key]                     // access (creates if absent!)
34. m.at(key)                  // access (throws if absent)
35. m.count(key)                // 1 or 0
36. m.find(key)                 // iterator
37. m.erase(key)                // remove key
38. m.size()                    // number of entries
39. for(auto& [k,v] : m)       // iterate

// stack
40. st.push(x)                 // 0(1)
41. st.pop()                   // 0(1), no return
42. st.top()                   // 0(1), peek

```

```
43. st.empty() // 0(1)

// queue
44. q.push(x) // 0(1)
45. q.pop() // 0(1), no return
46. q.front() / q.back() // 0(1)
47. q.empty() // 0(1)

// deque
48. dq.push_front(x) / push_back(x) // 0(1)
49. dq.pop_front() / pop_back() // 0(1)
50. dq.front() / dq.back() // 0(1)

// priority_queue
51. pq.push(x) // 0(log n)
52. pq.pop() // 0(log n)
53. pq.top() // 0(1)
54. priority_queue<int, vector<int>, greater<int>> // min-heap
```

Algorithms

cpp

```

// Sorting
55. sort(first, last) // 0(n log n)
56. sort(first, last, comparator) // custom sort
57. stable_sort(first, last) // preserves equal element order
58. partial_sort(first, middle, last) // sort first k elements
59. nth_element(first, nth, last) // partition around nth element
60. is_sorted(first, last) // check if sorted

// Searching
61. binary_search(first, last, val) // true/false, sorted required
62. lower_bound(first, last, val) // first >= val iterator
63. upper_bound(first, last, val) // first > val iterator
64. find(first, last, val) // linear search, returns iterator
65. count(first, last, val) // count occurrences
66. equal_range(first, last, val) // pair of (lower, upper) bound

// Min/Max
67. min(a, b) / max(a, b) // compare two values
68. min({a, b, c}) / max({a, b, c}) // compare initializer list
69. min_element(first, last) // iterator to min
70. max_element(first, last) // iterator to max
71. minmax_element(first, last) // pair of (min_it, max_it)
72. clamp(val, lo, hi) // C++17, clamps to [lo, hi]

// Math / Numeric
73. accumulate(first, last, init) // sum
74. iota(first, last, start) // fill with consecutive values
75. gcd(a, b) / lcm(a, b) // C++17 greatest common divisor
76. abs(x) // absolute value

// Permutations
77. next_permutation(first, last) // next permutation, returns false
78. prev_permutation(first, last) // previous permutation

// Modifying
79. reverse(first, last) // reverse in place
80. fill(first, last, val) // fill with value
81. copy(src_first, src_last, dst) // copy range
82. swap(a, b) // swap two values
83. rotate(first, middle, last) // rotate range
84. unique(first, last) // remove consecutive duplicates
85. remove(first, last, val) // remove value (use with erase!)

// Set Operations (sorted ranges)
86. set_union(...) // union of two sorted ranges

```



```
87. set_intersection(...) // intersection
88. set_difference(...) // difference
```

Bit Operations

```
cpp

89. __builtin_popcount(n) // count set bits (int)
90. __builtin_popcountll(n) // count set bits (long long)
91. __builtin_clz(n) // count leading zeros
92. __builtin_ctz(n) // count trailing zeros
93. __builtin_parity(n) // parity of bit count
```

Math

```
cpp

94. sqrt(x) // square root (double)
95. pow(x, y) // x^y (double) – use custom power for integers
96. ceil(x) / floor(x) // ceiling / floor
97. log(x) / log2(x) / log10(x) // natural / base-2 / base-10 log
98. INT_MAX / INT_MIN // 2^31-1 / -2^31
99. LLONG_MAX / LLONG_MIN // 2^63-1 / -2^63
100. numeric_limits<int>::max() // same as INT_MAX but generic
```

TIME COMPLEXITY CHEAT SHEET

$O(1)$	Constant:	Array index, hash map lookup, stack push/pop
$O(\log n)$	Log:	Binary search, balanced BST operations, fast power
$O(\sqrt{n})$	Sqrt:	Trial division, sqrt decomposition
$O(n)$	Linear:	Single loop, linear search, BFS/DFS
$O(n \log n)$	$N \log N$:	Merge sort, heap sort, most STL sorts, Dijkstra
$O(n \sqrt{n})$	$N * \sqrt{N}$:	Mo's algorithm, some offline algorithms
$O(n^2)$	Quadratic:	Nested loops, bubble sort, naive string search
$O(n^2 \log n)$		Some DP with binary search, sort + nested loop
$O(n^3)$	Cubic:	Floyd-Warshall, matrix multiplication, some DP
$O(n^4)$	Quartic:	4D DP (very rare)
$O(2^n)$	Exponential:	All subsets, bitmask DP ($n \leq 20$)
$O(n!)$	Factorial:	All permutations ($n \leq 11$)

Limits by n:

$n \leq 10 \quad \rightarrow O(n!), O(n^6)$ ok

$n \leq 15$	$\rightarrow O(2^n * n)$ ok (bitmask DP)
$n \leq 20$	$\rightarrow O(2^n)$ ok
$n \leq 100$	$\rightarrow O(n^4)$ ok
$n \leq 500$	$\rightarrow O(n^3)$ ok
$n \leq 5000$	$\rightarrow O(n^2)$ ok
$n \leq 10^5$	$\rightarrow O(n \log n)$ needed
$n \leq 10^6$	$\rightarrow O(n)$ or $O(n \log n)$ (tight)
$n \leq 10^8$	$\rightarrow O(n)$ needed (but tight)
$n \leq 10^{18}$	$\rightarrow O(\log n)$ or $O(\sqrt{n})$ needed

Memory limits:

```
int = 4 bytes    → 108 ints ≈ 400 MB
long long = 8 bytes → 108 LLs ≈ 800 MB
Typical limit: 256 MB → ~6 * 107 ints or ~3 * 107 long longs
```

ALGORITHM SELECTION CHEAT SHEET

Find shortest path:

- Unweighted graph $\rightarrow$ **BFS**
- Non-negative weights $\rightarrow$ **Dijkstra**
- Negative weights $\rightarrow$ **Bellman-Ford**
- All pairs $\rightarrow$ **Floyd-Warshall**
- DAG $\rightarrow$ **DP + Topological Sort**
- 0-1 weights $\rightarrow$ **0-1 BFS (deque)**

Range queries on array:

- Static, sum $\rightarrow$ **Prefix Sum** ($O(1)$ query)
- Static, min/max $\rightarrow$ **Sparse Table** ($O(1)$ query)
- Dynamic, sum/min/max $\rightarrow$ **Segment Tree** ($O(\log n)$)
- Dynamic, range updates + range queries $\rightarrow$ **Segment Tree with Lazy Propagation**
- Point updates + prefix sums $\rightarrow$ **Fenwick Tree (BIT)** ($O(\log n)$, simpler code)

Find connected components / cycle detection:

- Undirected graph $\rightarrow$ **DFS/BFS** or **DSU**
- Directed graph, detect cycle $\rightarrow$ **DFS with 3-color**
- Dynamic connectivity $\rightarrow$ **DSU**

Counting/optimization with constraints:

- Overlapping subproblems → **Dynamic Programming**
- Greedy works if you can prove it → **Greedy**
- $n \leq 20$, all subsets → **Bitmask DP**

String problems:

- Pattern matching → **KMP** or **Z-algorithm** ($O(n+m)$)
- Many patterns → **Aho-Corasick**
- Suffix queries → **Suffix Array**
- Palindromes → **Manacher's algorithm**

Sorting:

- General purpose → **std::sort** (introsort, $O(n \log n)$)
- Small integer range → **Counting Sort** ($O(n+k)$)
- Stable sort needed → **std::stable_sort**
- Find k-th element → **nth_element** ($O(n)$)

Tree problems:

- LCA → **Binary Lifting** ($O(n \log n)$ preprocess, $O(\log n)$ query)
- Subtree queries → **Euler Tour + Segment Tree**
- Path queries → **Heavy-Light Decomposition (HLD)**

Graph problems:

- Order tasks with dependencies → **Topological Sort**
- Minimum cost to connect all nodes → **MST (Kruskal/Prim)**
- Strongly connected components → **Kosaraju/Tarjan**
- Bridges/Articulation points → **DFS (Tarjan)**

Two quantities to optimize:

- Minimize maximum → **Binary Search on Answer + Greedy Check**
- Maximize minimum → **Binary Search on Answer + Greedy Check**

Pair/triple sum = target:

- Two sum in sorted array → **Two Pointers** ($O(n)$)
- Two sum with hash → **Hash Map** ($O(n)$)
- Three sum → **Sort + Two Pointers** ($O(n^2)$)

COMMON PITFALLS AND PRO TIPS

Avoid These Bugs:

```
cpp

// 1. Integer overflow
int a = 1e9, b = 1e9;
int c = a + b;          // OVERFLOW! Use long long
long long c = (long long)a + b; // CORRECT

// 2. Unsigned size() issues
vector<int> v;
for(int i = 0; i < v.size() - 1; i++) // if v is empty, v.size()-1 underflows!
for(int i = 0; i + 1 < (int)v.size(); i++) // CORRECT

// 3. Modular arithmetic
(a * b) % MOD          // a*b might overflow before %
(1LL * a * b) % MOD    // CORRECT: force long long multiplication

// 4. Array out of bounds
arr[n]                 // n is out of bounds if array is [0..n-1]!

// 5. Using map[] to check existence
if(mp[key] > 0) ...     // creates key with default value 0 if absent!
if(mp.count(key) && mp[key] > 0) ... // CORRECT

// 6. endl vs \n
cout << endl;          // flushes buffer every time - very slow in loops!
cout << "\n";          // just newline - much faster
```

Pro Tips:

```
cpp
```

```

// Structured bindings (C++17) for cleaner code
for(auto& [key, val] : mp) cout << key << " " << val << "\n";
auto [a, b] = myPair;

// Lambda comparators
sort(v.begin(), v.end(), [](int a, int b){ return a > b; });

// Initialize 2D array to -1
memset(dp, -1, sizeof(dp)); // faster than nested loops for static arrays

// Swap to make a <= b
if(a > b) swap(a, b);

// Infinite loop with answer
while(true){
    // ... find answer ...
    cout << ans << "\n";
    break;
}

// Read until end
int x;
while(cin >> x) { /* process */ }

// Check if a number is even/odd
if(n & 1) // odd
else     // even

// Multiply/divide by 2 fast
n << 1 // multiply by 2
n >> 1 // divide by 2

// Keep answer in bounds
ans = max(ans, val);
ans = min(ans, val);

```

PRACTICE ROADMAP

Week 1-2: Foundations

- Arrays, Strings, Vectors
- Sorting (std::sort), Binary Search
- Prefix Sums, Frequency Arrays

- **Practice:** Codeforces Div. 4 A-B problems, CSES Intro problems

Week 3-4: Core Techniques

- Two Pointers, Sliding Window
- Hash Maps, Sets
- Stacks (especially monotonic stack), Queues
- **Practice:** Codeforces Div. 3 A-C, CSES Sorting & Searching

Week 5-6: Recursion & Trees

- Recursion, Backtracking
- Binary Trees, BSTs
- **Practice:** Leetcode tree section, Codeforces Div. 3 C-D

Week 7-8: Graphs

- DFS, BFS
- Topological Sort, DSU
- Dijkstra, Bellman-Ford
- **Practice:** CSES Graph Algorithms, Codeforces Div. 2 B-C

Week 9-10: DP

- 1D DP, 2D DP
- Knapsack variants, LCS, LIS
- **Practice:** CSES Dynamic Programming, Codeforces DP problems

Week 11-12: Advanced

- Segment Trees, Lazy Propagation
- Priority Queues (Dijkstra, Prim)
- Greedy + proofs
- Bit Manipulation
- **Practice:** CSES Advanced, Codeforces Div. 1 A-B

This guide covers the essential 95% of what you need for competitive programming up to Codeforces Div. 2 rating ~1600-1800. For higher ratings, study HLD, Sqrt Decomposition, Advanced DP, Network Flow, and Number Theory in depth.
